

METRIQA

User Guide & QA Reference Manual

Software Deployment & License Management Platform

Document	Complete functional manual — admin web portal, mobile app and API
Audience	End users · QA engineers
Chapters	20 chapters plus introduction
Generated	July 28, 2026
Source	docs/user-guide/ — derived from the shipped codebase

For QA: this manual is the authoritative statement of expected behaviour. Every rule is derived from the shipped code, so a mismatch between this document and the running application is a defect in one of the two. Read chapter 20 — Known Gaps & Troubleshooting — before filing a bug.

Contents

METRIQA — User Guide & QA Reference Manual

- How to use this manual for testing
- Contents
- Conventions used

01 — System Overview

- What METRIQA is
- Components
- Ports and startup
- Scheduled jobs
- Security model
- Real-time updates
- External integrations
- Glossary

02 — Roles & Permissions

- The seven roles
- Primary role vs additional roles
- Navigation matrix (admin web)
- Route guards (admin web)
- In-page capability gates
- Self-action bans
- Account state

03 — Getting Started

- First-time setup (fresh environment)
- Signing in
- Sessions and token refresh
- The app shell
- Notifications
- Theme
- Your profile (/profile)

04 — Dashboard & Navigation

- Role dashboards
- KPI widget
- Additional-role task cards
- Live data refresh
- Expected behaviour summary

05 — Clients

- Fields
- The list
- Creating a client
- Editing
- Deleting
- Permissions
- Client status meaning
- Expected behaviour summary

06 — Software Products & Licenses

- Part A — Software Products
- Part B — Licenses

- Part C — NENPOS Clients (legacy import)
- Expected behaviour summary

07 — Installations (Jobs)

- Two views of the same page
- Job fields
- Data scoping
- The job lifecycle
- Admin UI
- Calendar
- Creating jobs
- Expected behaviour summary

08 — Job Orders & Payments

- The list page
- The editor — a 3-step wizard
- Saving — POST /api/job-orders (upsert)
- Document types & printing
- Converting a quotation
- Payments
- Job order statuses
- Expected behaviour summary

09 — Inventory

- Item fields
- Barcode lookup
- Manual stock adjustment
- The stock ledger
- Automatic deduction from job orders
- Low-stock indicator
- Permissions
- Expected behaviour summary

10 — Earnings & Withdrawals

- Part A — Earnings
- Part B — Withdrawals
- Worked example (use this as a test scenario)
- Expected behaviour summary

11 — KPI & Incentives

- KPI-eligible roles
- Scoring
- Default KPI definitions
- Manual override
- Managing KPI definitions
- Generating incentives
- Team KPI (Analytics page)
- Designer KPI — TMS Pro integration
- Expected behaviour summary

12 — Dev Projects

- Project fields
- Visibility
- The timer

Progress
Reports
The page
Supporting endpoints
Expected behaviour summary

13 — Financial Reports & Analytics

Collections summary
Outstanding balances
Client payment history
CSV export
The page
Analytics page (Super Admin)
Expected behaviour summary

14 — Leads & the Public Landing Page

The landing page
Download Leads (admin)
Permissions
Expected behaviour summary

15 — Settings & Administration

Company Profile
Users & Roles
Database Management
Audit Logs
Expected behaviour summary

16 — Mobile App (Beulah Field)

Authentication
Backend environment switcher
Tabs
Admin section guard
Push notifications
Expected behaviour summary

17 — Status & Enum Reference

UserRole
ClientStatus
ClientType
LicenseType (product)
LicenseStatus
JobStatus
JobOrderStatus
JobOrderType
DocType
DiscountType
PaymentMethod
EarningType
EarningStatus
WithdrawalMethod
WithdrawalStatus
IncentiveStatus
StockMovementReason
DevProjectStatus

DevReportStatus
Other value sets (not database enums)

18 — API & Permission Matrix

Auth
Users
Clients
Software Products
Licenses
NENPOS Clients
Jobs
Job Orders
Payments
Inventory
Earnings
Withdrawals
KPIs & Incentives
Dev Projects
Financial Reports
Notifications
Events (SSE)
Uploads
Company Profile
Audit Logs
Backups / Restore / Reset
Download Leads
Standard error shapes
Authorization test recipe

19 — QA Test Checklists

Test accounts to prepare
A — Authentication & session [03]
B — Roles & navigation [02]
C — Clients [05]
D — Products & licenses [06]
E — Installations [07]
F — Job orders & payments [08]
G — Inventory [09]
H — Earnings & withdrawals [10]
I — KPI & incentives [11]
J — Dev projects [12]
K — Financial reports [13]
L — Leads [14]
M — Settings & administration [15]
N — Mobile [16]
O — Cross-cutting
Smoke test (15 minutes)

20 — Known Gaps & Troubleshooting

Permission mismatches
Scoping gaps
Business-rule questions to confirm with the product owner
Incomplete features
Environment / infrastructure notes
Error message catalogue
Common UI error strings and what they mean

METRIQA — User Guide & QA Reference Manual

Complete functional manual for the **METRIQA** platform (internally still named SDLMP — Software Deployment & License Management Platform).

This manual is written to serve two audiences at once:

- **End users** — step-by-step instructions for every screen and workflow.
- **QA** — the authoritative statement of *expected behaviour*. Every rule in here is derived from the shipped code, so a mismatch between this manual and the running app is a defect in one of the two.

How to use this manual for testing

1. Start with [02 — Roles & Permissions](#) and create one test account per role. Almost every defect in this system is role-scoped.
2. Work a module chapter (04–16) top to bottom. Each chapter ends with an **Expected behaviour** table — those are the assertions to verify.
3. Use [17 — Status Reference](#) for the exact state machines. Illegal transitions must be rejected with the documented message.
4. Use [18 — API & Permission Matrix](#) when testing authorization directly against the API (bypassing the UI).
5. Run [19 — QA Test Checklists](#) as the regression pass.
6. Read [20 — Known Gaps & Troubleshooting](#) **before** filing bugs — several UI/API permission mismatches are already documented there.

Contents

#	Chapter	Covers
01	System Overview	Architecture, components, environments, glossary
02	Roles & Permissions	7 roles, primary vs additional roles, navigation matrix
03	Getting Started	Login, sessions, profile, notifications, theme
04	Dashboard & Navigation	Per-role dashboards, sidebar, live updates
05	Clients	Client records, codes, statuses, types
06	Products & Licenses	Software products, license generation, trials, activation, NENPOS import
07	Installations (Jobs)	Scheduling, assignment, proof of installation, calendar
08	Job Orders & Payments	3-step wizard, doc types, pricing, payments, printing
09	Inventory	Items, barcodes, stock ledger, auto-deduction
10	Earnings & Withdrawals	Earning types, balance formula, payout workflow
11	KPI & Incentives	KPI definitions, scoring, incentive tiers, TMS sync
12	Dev Projects	Project timer, progress, reports, review feedback
13	Financial Reports & Analytics	Collections, outstanding, client history, revenue
14	Leads & Landing Page	Public download form, email OTP, Finara leads
15	Settings & Administration	Company profile, users, backups, restore, reset, audit logs
16	Mobile App	Beulah Field — tabs, admin menu, proof capture, push
17	Status Reference	Every enum and state machine
18	API & Permission Matrix	Endpoint-by-endpoint role requirements
19	QA Test Checklists	Regression pass, per module
20	Known Gaps & Troubleshooting	Documented mismatches, error catalogue

Conventions used

- `SUPER_ADMIN` , `ASSIGNED` , `FINALIZED` — literal enum values as stored and as returned by the API.
- "Admin" in UI copy means the `SUPER_ADMIN` role; the sidebar shows the label **Admin** for it.
- Currency is Philippine Peso (₱) throughout; amounts are `Decimal(12,2)` in the database and serialized as **strings** in API responses.
- Paths like `/job-orders/order/:joId` are admin-web routes; paths like `POST /api/licenses` are backend endpoints.
- **Expected:** marks an assertion QA should verify.

01 — System Overview

What METRIQA is

METRIQA is a centralised platform for a software/hardware deployment business. It tracks the full chain from **client acquisition** → **job order** → **installation** → **license activation** → **payment collection** → **staff incentive payout**, plus internal development-project tracking and KPI-based performance bonuses.

Tagline shown in the sidebar: *Measure · Monitor · Reward.*

Components

Component	Technology	Location	Notes
API / backend	NestJS 11 + Prisma 6 + MySQL	<code>src/</code> , <code>prisma/</code>	All business logic and authorization
Admin web portal	React 19 + Vite + React Router + TanStack Query	<code>admin-web/</code>	Built to <code>admin-web/dist</code> , served by the API as static files
Mobile app ("Beulah Field")	React Native / Expo Router	<code>mobile/</code>	Field staff + a cut-down admin section
Public landing page	Part of admin-web (<code>/</code> when signed out)	<code>admin-web/src/pages/LandingPage.tsx</code>	Lead capture + app download

Request routing

- API is mounted under the global prefix `/api` .
- `/downloads/*` serves installer files (e.g. the mobile APK) from `<cwd>/downloads` with a forced `Content-Disposition: attachment` header.
- Everything else falls through to the built admin-web SPA (`admin-web/dist`), excluding `/api/*` and `/downloads/*` .

Expected: hitting a non-API, non-download path returns the SPA shell, and the SPA's catch-all route (`*`) redirects to `/` .

Ports and startup

Command	Effect
<code>npm run start:dev</code>	Sets <code>PORT=3002</code> and starts Nest in watch mode
<code>npm run start:prod</code>	<code>node dist/main</code> , port from <code>PORT</code> env, default 3000
<code>npm run dev</code>	Runs backend + admin-web Vite dev server concurrently
<code>npm run build</code>	Builds admin-web then compiles the Nest app
<code>npm run db:seed</code>	Creates the first Super Admin

On boot, `main.ts` runs `npx prisma migrate deploy` **before** creating the Nest app. If migrations fail the app still boots — the failure is logged, not fatal.

Expected: a schema-drifted database produces migration output in the logs and endpoints touching missing columns return 500 rather than the app crash-looping.

Scheduled jobs

Cron	Job	Effect
0 2 * * * (02:00 daily)	LicensesService.expireOverdueLicenses	ACTIVATED licenses whose expirationDate has passed become EXPIRED
0 3 * * * (03:00 daily)	BackupsService.scheduledBackup	Full SQL dump into backups/, keeps the newest 14, prunes the rest

Security model

- **Auth:** email + password (bcrypt, cost 12) → JWT access token + refresh token.
 - Access token default lifetime 15m (JWT_ACCESS_EXPIRES_IN).
 - Refresh token default lifetime 7d (JWT_REFRESH_EXPIRES_IN), stored in the users.refresh_token column as a bcrypt hash.
- **Authorization:** RolesGuard — a request passes when **any** of the user's roles (primary + additional) matches **any** role on the @Roles(...) decorator.
- **Rate limiting** (per IP, @nestjs/throttler, trust proxy = loopback):
 - Global: 300 requests / 60 s
 - POST /api/auth/login : 10 / 60 s
 - POST /api/download-leads/send-code : 5 / 60 s
 - POST /api/download-leads : 10 / 60 s
- **Validation:** global ValidationPipe with whitelist: true, transform: true, forbidNonWhitelisted: true. **Expected:** posting an unknown property returns 400, not a silent drop.
- **License signing:** activation issues an RS256 JWT signed with an RSA-4096 key pair generated on first run under keys/ (gitignored). Losing that key pair invalidates previously issued license tokens.
- **Audit trail:** a global interceptor records every POST/PUT/PATCH/DELETE (plus login/logout, including failed logins) into audit_logs, scrubbing password, refreshToken, token, secret, mfaSecret to *****.

Real-time updates

- **SSE** — GET /api/events streams two things:
 - per-user notifications (persisted in the notifications table first);
 - a broadcast "data changed" event on every mutation, carrying { resource, module, action, actor, actorRole }.
- The admin web uses these to show the notification bell badge and a red dot on sidebar items whose module changed.
- **Push** — Firebase Cloud Messaging, when FIREBASE_SERVICE_ACCOUNT_PATH (or FIREBASE_SERVICE_ACCOUNT) is configured. Without credentials, push is skipped silently and everything else keeps working.

External integrations

Integration	Env vars	Used by
TMS Pro (designer KPI points)	TMS_KPI_API_URL (default https://tmspro.up.railway.app), TMS_KPI_API_TOKEN	Settings → KPI Settings → Designer sync
Finara ERP (leads)	FINARA_API_URL (default https://finara.up.railway.app), FINARA_API_KEY	Download Leads → Finara tab
Resend (email OTP)	RESEND_API_KEY, RESEND_FROM	Landing page lead verification
Firebase FCM	FIREBASE_SERVICE_ACCOUNT_PATH / FIREBASE_SERVICE_ACCOUNT	Device push

Expected: every one of these degrades gracefully — a missing key produces a clear 400 with a human-readable reason (or, for Resend, falls back to accepting the lead with an unverified email), never a 500.

Glossary

Term	Meaning
Client	The paying business. Has a <code>clientCode</code> (<code>CLT-XXXXXXX</code>), type <code>SOFTWARE</code> or <code>ADVERTISING</code> .
Software Product	A sellable product + version with a license type and price.
License	A key issued to a client for a product. May be a trial (<code>TRIAL-XXXX-XXXX</code> , auto-generated) or a real provider key.
Job	An installation appointment: client + schedule date + optional installer + optional license.
Job Order (JO)	The commercial document for a job: sale price, line items, discount, payments. Can print as Job Order / Quotation / Invoice / Official Receipt.
Proof of Installation	Photos + optional signature + GPS + device info, captured by the installer.
Earning	A money credit to a staff member (<code>INSTALLATION</code> , <code>ACTIVATION</code> , <code>BONUS</code> , <code>COMMISSION</code>).
Withdrawal	A staff request to cash out available balance.
Incentive	A monthly KPI-derived bonus; mirrored 1:1 into an <code>Earning</code> of type <code>BONUS</code> .
Dev Project	An internal software project with a start/pause/stop timer and progress %.
NENPOS Client	Legacy pre-system client records, bulk-imported from Excel. Read/report only — not linked to licenses or jobs.
Download Lead	A visitor who filled the landing-page form to download the app.

02 — Roles & Permissions

The seven roles

Enum value	Sidebar label	Purpose
<code>SUPER_ADMIN</code>	Admin	Full access. The only role that can manage users, settings, backups, analytics.
<code>ADMIN_STAFF</code>	Staff	Operations back-office: clients, job orders, dev projects, products, installations, finance, leads.
<code>SALES_STAFF</code>	Sales	Client-facing sales: clients, job orders, installations, financial reports, own payroll.
<code>LIAISON</code>	Liaison	Field coordination: clients, project job orders, installations, own payroll.
<code>INSTALLER</code>	Installer	Field installation: own jobs, proof submission, own earnings/withdrawals.
<code>DEVELOPER</code>	Developer	Dev projects, license activation, own payroll.
<code>DESIGNER</code>	Designer	Own payroll + KPI only (KPI points sourced from TMS Pro). Can view job orders.

Primary role vs additional roles

A user has:

- **role** — the *primary* role. It decides **which dashboard** they see and **which navigation set** is used as the base.
- **additionalRoles** — zero or more extra roles stored in `user_role_assignments`.
- **roles** — the merged array (`[primary, ...additional]`) returned in the login response and in `GET /api/users/me`. This is what the backend `RolesGuard` checks.

Rules enforced by the API:

- Creating or editing a user filters the primary role out of `additionalRoles` automatically — the same role can never appear twice.
- Changing the primary role via `PATCH /api/users/:id/primary-role` deletes that role from `additionalRoles` first.
- Sending `additionalRoles` on `PATCH /api/users/:id` **replaces** the whole set (it deletes all, then creates the supplied list).

Expected: a user with primary `SALES_STAFF` + additional `INSTALLER` sees the Sales dashboard, but passes any `@Roles(INSTALLER)` endpoint check.

Merged navigation

The sidebar starts from the primary role's nav list, then appends — under a section header named after each extra role — any links that role adds and the primary role doesn't already have. Duplicate paths are never repeated.

Expected: primary `DESIGNER` + additional `INSTALLER` shows the Designer nav, then an **Installer** section header containing *My Jobs* only (Dashboard, Earnings and Withdrawals are already present).

Navigation matrix (admin web)

✓ = link appears in the sidebar for that primary role.

Route	Label	<code>SUPER_ADMIN</code>	<code>ADMIN_STAFF</code>	<code>SALES_STAFF</code>	<code>LIAISON</code>	<code>INSTALLER</code>	<code>DEVELOPER</code>	<code>DESIGNER</code>
<code>/</code>	Dashboard	✓	✓	✓	✓	✓	✓	✓
<code>/clients</code>	Clients	✓	✓	✓	✓			
<code>/job-orders/software</code>	Project Job Orders / Project JO	✓	✓	✓	✓			

Route	Label	SUPER_ADMIN	ADMIN_STAFF	SALES_STAFF	LIAISON	INSTALLER	DEVELOPER	DESIGNER
/dev-projects	Dev Projects	✓	✓				✓	
/products	Software Products	✓	✓					
/licenses	Licenses	✓					✓	
/jobs	Installations / My Jobs	✓	✓	✓	✓	✓		
/earnings	Earnings / My Earnings	✓	✓	✓	✓	✓	✓	✓
/withdrawals	Withdrawals	✓	✓	✓	✓	✓	✓	✓
/financial-reports	Financial Reports	✓	✓	✓				
/download-leads	Download Leads	✓	✓					
/audit-logs	Audit Logs		✓					
/analytics	Analytics	✓						
/settings	Settings	✓						
/profile	Profile Settings	✓	✓	✓	✓	✓	✓	✓

Note: Super Admin has no *Audit Logs* sidebar link because audit logs are a tab inside **Settings → Audit Logs**.

Route guards (admin web)

Routes are additionally guarded by `RequireAuth roles={ [... ] }`. A user who is signed in but lacks the role is redirected rather than shown the page.

Route	Allowed roles
/, /earnings, /withdrawals, /profile	all 7
/dev-projects	DEVELOPER, ADMIN_STAFF, SUPER_ADMIN
/jobs	SUPER_ADMIN, INSTALLER, ADMIN_STAFF, LIAISON, SALES_STAFF
/job-orders/software	SUPER_ADMIN, ADMIN_STAFF, LIAISON, SALES_STAFF
/job-orders/:jobId, /job-orders/order/:joId	above + DESIGNER
/financial-reports	SUPER_ADMIN, ADMIN_STAFF, SALES_STAFF
/licenses	SUPER_ADMIN, DEVELOPER, ADMIN_STAFF
/clients	SUPER_ADMIN, ADMIN_STAFF, SALES_STAFF, LIAISON
/products	SUPER_ADMIN, ADMIN_STAFF
/download-leads, /audit-logs	SUPER_ADMIN, ADMIN_STAFF
/users, /analytics, /settings	SUPER_ADMIN
/login, /developers	public (no auth)

⚠ Two of these front-end guards are **wider** than the matching API guard. See [20 — Known Gaps](#).

In-page capability gates

Beyond routing, several pages hide actions by role:

Page	Capability	Roles
Earnings	Approve / Mark paid	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>
Earnings	Allocate (create) an earning	<code>SUPER_ADMIN</code> only
Withdrawals	Approve / Reject / Release	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>
Job Order → Payments	Void a payment	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>
Installations	Installer view (my jobs only)	has <code>INSTALLER</code> and primary role is not an admin-type role

Self-action bans

These are enforced server-side and must be verified by QA:

- You **cannot approve or pay your own earning** → `403 You cannot approve or pay out your own earning.`
- You **cannot process your own withdrawal** (approve/reject/release) → `403 You cannot process your own withdrawal request.`

Expected: a Super Admin who is also the recipient gets 403 on both; a *different* admin succeeds.

Account state

- `isActive = false` blocks login (`401 Invalid credentials`) **and** invalidates existing access tokens — `JwtStrategy.validate` re-reads the user on every request and throws `401 Account is inactive or no longer exists` .

Expected: deactivating a signed-in user kicks them out on their next API call, not only at next login.

03 — Getting Started

First-time setup (fresh environment)

1. Point `DATABASE_URL` at a MySQL instance in `.env`.
2. `npx prisma migrate dev` — creates the schema.
3. `npm run db:seed` — creates the first Super Admin.
 - Email: `SEED_SUPER_ADMIN_EMAIL`, default `admin@sdtmp.local`
 - Password: `SEED_SUPER_ADMIN_PASSWORD`, default `ChangeMe123!`
 - Full name is set to `Super Admin`.
 - Re-running the seed on an existing account is a **no-op** and logs `Super admin already exists`.
4. Start the API (`npm run start:dev` → port 3002) and the portal.

Expected: the seed never overwrites an existing user's password.

On first API start, an RSA-4096 key pair is generated under `keys/` for license signing. Back it up — regenerating it invalidates previously issued license tokens.

Also seeded automatically at startup: **KPI definitions** for every KPI role (see [11 — KPI & Incentives](#)). Seeding runs once per role, sequentially, and is skipped when definitions already exist.

Signing in

Route: `/login`.

Field	Rules
Email	Must match an existing user exactly (unique index)
Password	bcrypt-compared against the stored hash

On success the API returns `{ accessToken, refreshToken, user }` where `user` contains `id`, `email`, `role` (primary), `roles` (merged array) and `fullName`.

Failure cases — **all return the same generic message** so accounts cannot be enumerated:

Situation	Response
Unknown email	<code>401 Invalid credentials</code>
Wrong password	<code>401 Invalid credentials</code>
Account deactivated (<code>isActive = false</code>)	<code>401 Invalid credentials</code>
More than 10 attempts in 60 s from one IP	<code>429 ThrottlerException: Too Many Requests</code>

Expected: every login attempt — success *and* failure — writes an audit-log row. Failures are recorded as `Failed User Login (401)`.

MFA: the `users` table carries `mfaEnabled` / `mfaSecret` columns and the Users screen displays an MFA column, but **no OTP step is implemented in the login flow**. Treat MFA as not-yet-shipped. See [20 — Known Gaps](#).

Sessions and token refresh

- Access token lifetime: **15 minutes** by default.
- Refresh token lifetime: **7 days** by default; only the bcrypt **hash** is stored.
- `POST /api/auth/refresh` issues a *new pair* and rotates the stored hash.

- `POST /api/auth/logout` sets `users.refresh_token = NULL`.

Expected behaviour

Scenario	Result
Access token expires mid-session	Client silently refreshes and the action completes
Refresh with an already-rotated token	<code>401 Access denied</code>
Refresh after logout	<code>401 Access denied</code>
Logout while the network is down (web)	Local session is cleared anyway and the user lands on <code>/login</code>

The app shell

After login every role lands on `/` (Dashboard) inside the same shell:

- **Sidebar (left)** — brand block (METRIQA logo + *Measure · Monitor · Reward*), role-based navigation, then the user block: avatar initials, full name, role label, **Profile Settings**, **Log out**.
- **Header (top)** — sidebar collapse toggle (desktop) or hamburger (mobile), **notification bell**, **theme toggle**.
- **Content** — the routed page, with a fade/slide page transition.
- **Floating timer widget** — bottom of the viewport, only while a dev project timer is running.

Sidebar behaviour

Viewport	Behaviour
> 900 px	Fixed sidebar, 256 px expanded / 74 px icon-only rail. Collapse state is persisted in <code>localStorage</code> under <code>sdmp-sidebar-collapsed</code> .
≤ 900 px	Off-canvas drawer with a dimmed backdrop; opens via the hamburger, auto-closes on route change and on backdrop click.

Expected: in rail mode, labels are hidden, icons are centred, and hovering a link shows its label as a native tooltip.

Unread indicators

A small red dot appears on a sidebar link when its module has changed since you last visited it (driven by the SSE broadcast). Opening that route clears the dot.

Expected: the dot never shows on the route you are currently viewing.

Notifications

The bell in the header lists your persisted notifications, newest first (up to 50).

Action	Endpoint
List	<code>GET /api/notifications</code>
Unread badge count	<code>GET /api/notifications/unread-count</code>
Mark one read	<code>PATCH /api/notifications/:id/read</code>
Mark all read	<code>PATCH /api/notifications/read-all</code>

Events that generate a notification:

Event type	Recipient	Trigger
<code>job_assigned</code>	The installer	A job is created with, or reassigned to, that installer
<code>withdrawal_status</code>	The requester	Withdrawal approved / rejected / released
<code>dev_report_tagged</code>	The tagged admin	A developer posts a report tagging them

Event type	Recipient	Trigger
dev_report_feedback	The report author	The tagged admin (or a Super Admin) replies

Rules:

- Marking someone else's notification read returns `403 Not your notification`.
- Marking an already-read notification is idempotent (returns it unchanged).
- Notification delivery failures are swallowed — **a failed notification must never fail the underlying action** (e.g. job assignment still succeeds).

Expected: with FCM unconfigured, the in-app bell still works; only device push is skipped.

Theme

The header toggle switches light/dark. The choice persists across reloads.

Your profile (/profile)

Three sections: **Account Info**, **Change Password**, and read-only account details.

Field	Rules
Full name	Free text
Email address	Must be unique → <code>409 Email already in use by another account</code>
Phone / contact number	Optional; empty string is stored as <code>NULL</code>
Current password	Required whenever a new password is supplied
New password / Confirm new password	Must match (client-side check)

Server rules:

- New password without current password → `400 Current password is required to set a new one`
- Wrong current password → `401 Current password is incorrect`
- Passwords are re-hashed with bcrypt cost 12.

Expected: you cannot change your own **role**, **base bonus**, or **active status** from `/profile` — those are Super Admin operations on `/users`.

04 — Dashboard & Navigation

Route: `/` (all roles). The heading always reads **"Welcome back, {first name}"**.

Beside the heading sit the **Quick Actions** buttons (visible on every role's dashboard):

Button	Action
+ New Client	Opens the <i>New Client</i> dialog with a pre-generated <code>CLT-XXXXXXX</code> code (regenerate with ↻)
+ New Quotation	Navigates to <code>/job-orders/order/new?doc=QUOTATION</code> — a standalone quotation, no installation job required
+ Create Job Order	Navigates to <code>/job-orders/software</code>

Expected: the New Client dialog resets its form and regenerates the code every time it is opened, and again after a successful save.

Role dashboards

The dashboard body is chosen by the user's **primary role**.

Super Admin

Summary cards:

Card	Value
Total Revenue	₹ sum of FINALIZED job-order sale prices (all time)
Monthly Growth	% change of this month's vs last month's finalized revenue, signed
Active Clients	<code>ACTIVE</code> clients / all clients
Activated Licenses	<code>ACTIVATED</code> licenses / all licenses
Open Installation Jobs	non-completed jobs / all jobs
Pending Withdrawals	<code>PENDING</code> withdrawals / all withdrawals

Charts: **Licenses by Status** (Activated / Pending / Expired) and **Installation Jobs** (Active = `ASSIGNED` + `ON_GOING`, Waiting = `WAITING_ACTIVATION`, Completed).

Also present: a **calendar** of scheduled installations. Clicking a date opens a dialog titled with the full date (e.g. *Monday, 3 August 2026*) listing that day's jobs; an empty date shows **"No jobs scheduled on this date."**

Installer

Cards: **Active Jobs, Awaiting Activation, Completed Jobs, Pending Earnings**. Chart: *Job Status Distribution — Your job breakdown*. Plus the same calendar, scoped to the installer's own jobs.

Developer

Cards: **Pending Activations, Activated by Me, Pending Earnings, Pending Withdrawals**. Charts: *License Status* (Pending / Activated by Me / Other) and *Earnings & Withdrawals*.

Designer

Cards: **Pending Earnings, Approved Earnings, Pending Withdrawals**. Designers have no operational modules — their scorecard comes from TMS Pro.

Liaison

Cards: **Active Clients, Pending Earnings, Approved Earnings, Pending Withdrawals**.

Admin Staff

Cards: **Active Jobs**, **Total Earnings**, **Pending Withdrawals**.

Sales Staff

Cards: **Active Clients**, **Total Job Orders**, **Sales Pipeline**, **Available Balance**. Charts: *Job Orders by Status* (Draft / Finalized / On-Going / Completed / Cancelled) and *Client Portfolio* (Active / Expired / Suspended / Cancelled). Below them, a **Recent Job Orders** list.

KPI widget

Every KPI-eligible role sees a **Performance scorecard** card with the current month's KPI table (Name, Actual, Target, Weight, Score) and the incentive estimate. See [11 — KPI & Incentives](#).

Additional-role task cards

If the signed-in user holds extra roles, the dashboard appends a task card per extra role summarising that role's outstanding work — installer job counts, developer pending activations and dev projects.

Expected: a user whose only role is primary sees no additional-role cards.

Live data refresh

Any mutation anywhere in the system broadcasts an SSE event. The dashboard's queries are invalidated on the relevant events, so counts update without a manual reload.

Expected: with two browsers open (admin + installer), an installer submitting proof updates the admin's *Open Installation Jobs* card without a refresh.

Expected behaviour summary

#	Assertion
4.1	Each of the 7 roles renders its own dashboard variant; no role sees another's cards
4.2	<i>Total Revenue</i> counts only FINALIZED job orders — a DRAFT or COMPLETED order must not move it
4.3	<i>Monthly Growth</i> shows +100.0% when the previous month was ₹0 and this month is positive, 0.0% when both are ₹0
4.4	Calendar dialog title is the localised long date (en-PH)
4.5	Quick Action <i>New Quotation</i> opens the JO editor pre-set to document type Quotation
4.6	Card denominators (x / y) always use the full unfiltered list length

05 — Clients

Route: `/clients` · Roles: `SUPER_ADMIN` , `ADMIN_STAFF` , `SALES_STAFF` , `LIAISON`

A **Client** is the paying business. Everything commercial hangs off it: licenses, installation jobs and job orders.

Fields

Field	Required	Rules
Client code	✓	Unique. Format <code>CLT-XXXXXXX</code> using an ambiguity-free alphabet (<code>ABCDEFGHIJKLMNPQRSTUVWXYZ23456789</code> — no <code>I</code> , <code>O</code> , <code>0</code> , <code>1</code>). Generated client-side; regenerate with the ↻ button.
Business name	✓	Displayed everywhere the client is referenced
Owner name	✓	
Contact no.	✓	
Email		Optional
Address		Optional
Client type	✓	<code>SOFTWARE</code> (default) or <code>ADVERTISING</code>
Status	✓	<code>ACTIVE</code> (default), <code>EXPIRED</code> , <code>SUSPENDED</code> , <code>CANCELLED</code>

Expected: a duplicate `clientId` is rejected by the database unique index.

The list

Columns: **Code** · **Business** · **Owner** · **Contact** · **Email** · **Type** · **Status** · **Edit/Delete**.

- Sorted by creation date, newest first.
- A search box filters the loaded rows client-side; no match shows **"No matches."**
- An empty dataset shows **"No clients yet."**
- A failed load shows **"Failed to load clients."**

The list can be filtered by type via `GET /api/clients?type=SOFTWARE|ADVERTISING` .

Creating a client

Two entry points, same dialog:

1. Dashboard → **+ New Client**
2. Clients page → **New Client**

Also available inline while building a job order — see [08 — Job Orders](#).

Editing

Opens the same form pre-filled. On failure the dialog shows **"Could not update the client. Try again."**

Deleting

Roles: `SUPER_ADMIN` , `ADMIN_STAFF` .

⚠️ **Delete is a hard delete with no cascade guard.** A client that still has licenses, jobs or job orders will fail on a foreign-key constraint and surface as a 500-level error. Delete only clients with no downstream records.

Expected (QA): attempt to delete a client that owns a license — record the exact error returned. This is a known rough edge, tracked in [20](#) — [Known Gaps](#).

Permissions

Operation	Endpoint	Roles
Create	<code>POST /api/clients</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>
List	<code>GET /api/clients</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code> , <code>LIAISON</code> , <code>SALES_STAFF</code> , <code>DESIGNER</code>
Get one	<code>GET /api/clients/:id</code>	same as list (includes <code>licenses</code> and <code>jobs</code>)
Update	<code>PATCH /api/clients/:id</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>
Delete	<code>DELETE /api/clients/:id</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>

Note that `DESIGNER` can **read** clients (needed by the job-order view) but not write them, and has no `/clients` sidebar link or route access — designers reach client data only through a job order.

Client status meaning

Status	Meaning
<code>ACTIVE</code>	Live account, counts toward the <i>Active Clients</i> dashboard card
<code>EXPIRED</code>	Subscription lapsed
<code>SUSPENDED</code>	Temporarily disabled
<code>CANCELLED</code>	Terminated

Client status is **informational** — it is not automatically driven by license expiry, and it does not block creating jobs or job orders for that client.

Expected: setting a client to `CANCELLED` does not cancel their licenses, jobs or job orders.

Expected behaviour summary

#	Assertion
5.1	Client code is unique; a duplicate is rejected
5.2	Generated codes never contain I, O, 0 or 1
5.3	<code>LIAISON</code> and <code>SALES_STAFF</code> can open <code>/clients</code> but get 403 on create/update/delete
5.4	Search filters across code, business, owner, contact and email
5.5	<code>GET /api/clients/:id</code> returns the client with its <code>licenses[]</code> and <code>jobs[]</code>
5.6	Every create/update/delete writes an audit-log row (<code>Created Client</code> / <code>Updated Client</code> / <code>Deleted Client</code>)

06 — Software Products & Licenses

Part A — Software Products

Route: `/products` · Roles: `SUPER_ADMIN` , `ADMIN_STAFF` (route) — but **write operations are `SUPER_ADMIN` only.**

A product is a sellable software package + version.

Field	Required	Rules
Product name	✓	
Version	✓	Free text (e.g. <code>2.4.1</code>)
License type	✓	<code>SUBSCRIPTION_MONTHLY</code> · <code>SUBSCRIPTION_ANNUAL</code> · <code>LIFETIME</code>
Price	✓	Number ≥ 0, <code>Decimal(12,2)</code>
Maintenance fee		Optional, number ≥ 0

Operation	Endpoint	Roles
Create	<code>POST /api/software-products</code>	<code>SUPER_ADMIN</code>
List / get	<code>GET /api/software-products[:id]</code>	any authenticated user
Update	<code>PATCH /api/software-products/:id</code>	<code>SUPER_ADMIN</code>
Delete	<code>DELETE /api/software-products/:id</code>	<code>SUPER_ADMIN</code>

Expected: `ADMIN_STAFF` can open `/products` and see the list, but every create/edit/delete returns **403**.

Empty state: **"No matches."** after filtering; **"Failed to load products."** on error.

Part B — Licenses

Route: `/licenses` · Sidebar for `SUPER_ADMIN` and `DEVELOPER` .

The page has **two tabs**:

- 1. **Licenses** — real license records
- 2. **NENPOS Clients** — imported legacy records (see Part C)

License record

Field	Notes
License key	Unique. Either a provider-issued key you paste in, or an auto-generated trial key
Client	Required, must exist
Software product	Required, must exist
Status	<code>PENDING</code> → <code>ACTIVATED</code> → (<code>EXPIRED</code> <code>SUSPENDED</code>)
Is trial / Trial days	Trial length 1–365, default 30
Activation date / Expiration date	Set at activation
Activated by	The developer who activated it
Hardware fingerprint	<code>{ cpu, disk, mac }</code> captured at activation

Field	Notes
License token	RS256 JWT signed with the RSA-4096 private key

Generating a license — `POST /api/licenses` (`SUPER_ADMIN`)

Two modes:

Full license (`isTrial` `false/omitted`)

- `licenseKey` is **required** → otherwise `400 License key is required for a non-trial license`
- Duplicate key → `409 A license with this key already exists`
- `expirationDate` optional
- Created with status `PENDING`

Trial license (`isTrial: true`)

- The server generates the key itself in the form `TRIAL-XXXX-XXXX` using the ambiguity-free alphabet (no l, O, 0, 1). Any `licenseKey` you send is ignored.
- `trialDays` defaults to **30**; validated to 1–365.
- `expirationDate` is stored as **null** — the trial window only starts at activation.
- Created with status `PENDING`.
- Key generation retries up to 5 times on a collision, then `500 Could not generate a unique trial license key`.

Missing client or product → `404 Client {id} not found` / `404 Software product {id} not found`.

Activating a license — `PATCH /api/licenses/:id/activate` (`DEVELOPER` only)

The activation dialog collects the machine's hardware fingerprint:

Field	Required
CPU identifier	✓
Disk serial	✓
MAC address	✓

On activation the API:

1. Rejects an already-activated license → `409 License is already activated`.
2. Sets `activationDate = now`.
3. Computes `expirationDate`:
 - **Trial:** `now + trialDays × 24 h`
 - **Non-trial:** keeps whatever `expirationDate` was set at generation (may be null = perpetual)
4. Signs an **RS256 JWT** containing `licenseId`, `licenseKey`, `clientId`, `productId`, `fingerprint`; the JWT's `exp` matches the expiration date when one exists.
5. Stores status `ACTIVATED`, `activatedById`, the fingerprint and the token.

Expected:

- A `SUPER_ADMIN` calling activate gets **403** — only `DEVELOPER` may activate.
- Activating a 7-day trial sets `expirationDate` to exactly 7 days after activation, not after generation.
- The issued token verifies against `keys/license-public.pem`.

Suspending — `PATCH /api/licenses/:id/suspend` (`SUPER_ADMIN`)

Sets status to `SUSPENDED`. The signed token is **not** revoked — suspension is a record-keeping state.

Editing — PATCH /api/licenses/:id (SUPER_ADMIN)

Designed for the **trial** → **full upgrade**: record the real provider key once the client pays.

Rules:

Rule	Result
Setting <code>isTrial: true</code> on a license that is not <code>PENDING</code>	400 An activated license cannot be changed back to a trial
New <code>licenseKey</code> already used by another license	409 A license with this key already exists
Converting trial → full	<code>trialDays</code> and <code>expirationDate</code> are both cleared to <code>null</code>
Staying a trial	<code>trialDays</code> = supplied value, else existing, else 30
Activation state and the signed token	Left untouched

Expected: upgrading an *activated* trial to a full license keeps status `ACTIVATED` , keeps `activationDate` and the original token, but clears the trial expiry — the license becomes perpetual on the record.

Automatic expiry

Nightly at **02:00**, every `ACTIVATED` license whose `expirationDate` is in the past flips to `EXPIRED` . The count is logged.

Expected: the sweep never touches `PENDING` or `SUSPENDED` licenses, and never touches licenses with a null `expirationDate` .

The list

Columns include License Key, Client, Product, Status, Activated, Expires, and the action buttons. Filters: a search box plus an **All statuses** dropdown (`ACTIVE` / `EXPIRED` / `SUSPENDED` / `CANCELLED` labels appear in the client filter; license statuses filter the license rows).

Empty/error states: **"No licenses match your search."** / **"Failed to load licenses."**

Permissions

Operation	Endpoint	Roles
Generate	POST /api/licenses	SUPER_ADMIN
List	GET /api/licenses	SUPER_ADMIN , DEVELOPER
Get one	GET /api/licenses/:id	SUPER_ADMIN , DEVELOPER
Activate	PATCH /api/licenses/:id/activate	DEVELOPER
Suspend	PATCH /api/licenses/:id/suspend	SUPER_ADMIN
Edit	PATCH /api/licenses/:id	SUPER_ADMIN

⚠ The admin-web route `/licenses` also admits `ADMIN_STAFF` , but `GET /api/licenses` does not. See [20 — Known Gaps](#).

Part C — NENPOS Clients (legacy import)

Second tab on the Licenses page. These are pre-system client records kept for reference; they are **not** linked to clients, licenses or jobs.

Column	Excel header
Client ID	Client ID — auto-generated as <code>NPC-XXXXXXX</code> when blank
Client Name	Client Name — required
Start Date	Start Date
Expiry Date	Expiry Date

Column	Excel header
License	License
Status	Status — defaults to ACTIVE
Installer	Installer
Notes	Notes
Address	Address

Workflow:

1. **Download template** → `nenpos_clients_template.csv` with one example row (`NPC-ABC123` , `Juan dela Cruz Store` , ...).
2. Fill it in and **Upload** the Excel/CSV file → `POST /api/nenpos-clients/upload` . Only the **first sheet** is read. Dates accept Excel serial numbers, `Date` values, or parseable date strings; unparseable values become `null` .
3. The response reports `{ imported: n }` .

Manual add/edit is also available via the **Add NENPOS Client** dialog.

Operation	Endpoint	Roles
List / create / upload / update / delete one	<code>/api/nenpos-clients...</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>
Delete all	<code>DELETE /api/nenpos-clients</code>	<code>SUPER_ADMIN</code>

Empty state: "**No NENPOS client records yet. Download the template, fill it in, and upload your Excel file.**"

Expected: a row with a blank Client Name is rejected (`400 Client Name is required.`); a row with a blank Client ID gets an auto-generated `NPC-` code.

Expected behaviour summary

#	Assertion
6.1	Trial keys always match <code>TRIAL-[A-Z2-9]{4}-[A-Z2-9]{4}</code> and never contain l, O, 0, 1
6.2	A trial license has <code>expirationDate = null</code> until it is activated
6.3	Trial expiry = activation time + <code>trialDays</code> days
6.4	Only <code>DEVELOPER</code> can activate; re-activating returns 409
6.5	Trial → full conversion clears <code>trialDays</code> and <code>expirationDate</code> , preserves <code>ACTIVATED</code> status and the token
6.6	Full → trial conversion is blocked unless the license is still <code>PENDING</code>
6.7	The 02:00 sweep expires only <code>ACTIVATED</code> licenses with a past expiry
6.8	Product create/update/delete is <code>SUPER_ADMIN</code> only, even though the page is reachable by <code>ADMIN_STAFF</code>
6.9	NENPOS import reads only the first worksheet and reports the imported count

07 — Installations (Jobs)

Route: `/jobs` · Roles: `SUPER_ADMIN` , `ADMIN_STAFF` , `SALES_STAFF` , `LIAISON` , `INSTALLER`

A **Job** is one scheduled installation visit.

Two views of the same page

The page renders one of two views:

View	Shown when	Heading
Installer view	The user has the <code>INSTALLER</code> role and their <i>primary</i> role is not an admin-type role	My Jobs
Admin view	Otherwise	Installations

An admin who also holds `INSTALLER` as an additional role still gets the admin view.

Empty states: **"No jobs assigned to you yet."** (installer) / **"No installations scheduled yet."** (admin). Errors: **"Failed to load your jobs."** / **"Failed to load installations."**

Job fields

Field	Required	Notes
Client	✓	
Schedule date	✓	
Installer		Optional at creation — the job can be scheduled unassigned (shown as Unassigned)
License		Optional link to a specific license
Status	✓	Defaults to <code>ASSIGNED</code> when an installer is set at creation
Remarks		Free text

Data scoping

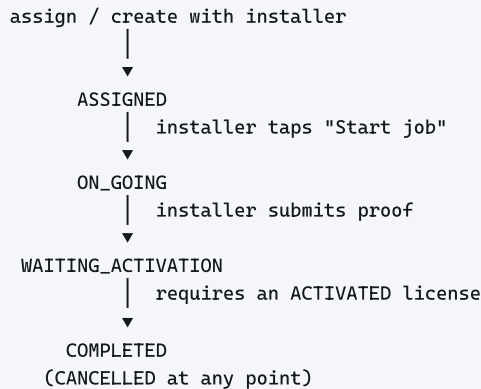
`GET /api/jobs` filters to `installerId = you` **only when your *primary* role is `INSTALLER`** . Everyone else receives all jobs.

Two consequences QA should test explicitly:

- A user whose primary role is e.g. `SALES_STAFF` with `INSTALLER` as an *additional* role sees **all** jobs — consistent with the admin view they get in the UI.
- `GET /api/jobs?mine=true` has **no effect** for anyone whose primary role is not `INSTALLER` ; the full list is still returned. See [20 — Known Gaps](#).

Expected: a pure `INSTALLER` cannot see another installer's jobs through the API, even by removing UI filters.

The job lifecycle



Assigning an installer — PATCH /api/jobs/:id/assign

Roles: SUPER_ADMIN, ADMIN_STAFF.

Sets `installerId` and forces status back to `ASSIGNED`, then sends the installer a "New installation job assigned" notification with the body "You've been assigned an installation job for {client business name}." and a deep link to `/jobs`.

Expected: reassigning an `ON_GOING` job resets it to `ASSIGNED` and notifies the new installer.

Changing status — PATCH /api/jobs/:id/status

Roles: **INSTALLER only.**

- An installer who is not the job's assignee gets `403 You are not assigned to this job`.
- Moving to `COMPLETED` requires an activated license. The check accepts **either**:
 - the job's directly linked license is `ACTIVATED`, or
 - the job's client has **at least one** `ACTIVATED` license.
- Otherwise: `400 Cannot complete this job: the license has not been activated yet. Please wait for the license to be activated before completing the task.`
- `remarks` is optional; omitting it preserves the existing remarks.

Expected: a `SUPER_ADMIN` calling this endpoint gets 403. Job completion is an installer action only.

Submitting proof of installation — POST /api/jobs/:id/proof

Roles: `INSTALLER`, and only for their own job (else `403 You are not assigned to this job`).

Captured:

Field	Required	Notes
Installation photos	✓ (array)	Uploaded first via <code>POST /api/uploads/images</code>
Client signature		A photo URL
GPS latitude / longitude		<code>Decimal(10,7)</code>
Device info		JSON blob
Captured at	auto	Set to now

Behaviour:

- The proof record is **upserted** — resubmitting overwrites the previous proof and refreshes `capturedAt`. There is exactly one proof per job (`job_id` unique).
- After a successful submit the job moves to `WAITING_ACTIVATION`, regardless of its previous status.

Expected: submitting proof twice leaves one proof row with the latest data and the job in `WAITING_ACTIVATION` .

Image uploads

`POST /api/uploads/images` (any authenticated user):

- Field name `files` , **max 10 files** per request
- **Max 10 MB** per file
- Allowed MIME types: `image/jpeg` , `image/png` , `image/gif` , `image/webp` — anything else → `400` Only image files (JPEG, PNG, GIF, WEBP) are allowed
- Files are stored with a random UUID name; the response is `{ urls: ["/api/uploads/files/<uuid>.<ext>", ...] }`
- `GET /api/uploads/files/:filename` serves them (no auth guard on the read)

Admin UI

Admins see, per job: **Client · Schedule · Installer · Status · Remarks**, with:

- **Assign** — pick an installer
- **Submitted Proof** panel — photos, signature, GPS. When nothing has been submitted: **"No proof submitted for this job yet."**; with a proof but no images: **"No photos attached."**

Installers see **Start job**, **Submit proof** and **Mark complete** actions plus the proof form (photos, optional signature photo, optional GPS lat/long).

Calendar

Two endpoints back the dashboard/page calendar:

Endpoint	Returns
<code>GET /api/jobs/calendar/month?month=&year=</code>	<code>id</code> , <code>scheduleDate</code> , <code>jobStatus</code> for jobs in that calendar month, ascending
<code>GET /api/jobs/calendar/day?date=</code>	Full job records (client, installer, license, proof) for that day

Roles for both: `SUPER_ADMIN` , `INSTALLER` , `ADMIN_STAFF` , `LIAISON` , `SALES_STAFF` .

Note: the calendar endpoints are **not** installer-scoped — they return all jobs for the period. Flagged in [20 — Known Gaps](#).

Creating jobs

Path	How
<code>POST /api/jobs</code>	<code>SUPER_ADMIN</code> only
Converting a standalone quotation	<code>POST /api/job-orders/:id/convert</code> creates the job and links it — see 08

Expected behaviour summary

#	Assertion
7.1	Installers see only their own jobs from <code>GET /api/jobs</code>
7.2	Only <code>INSTALLER</code> can change job status or submit proof; wrong installer → <code>403</code>
7.3	<code>COMPLETED</code> is blocked with the documented message unless the job's license — or <i>any</i> license of that client — is <code>ACTIVATED</code>
7.4	Submitting proof always sets the job to <code>WAITING_ACTIVATION</code>

#	Assertion
7.5	Re-submitting proof upserts (one proof row per job) and updates <code>capturedAt</code>
7.6	Uploading a PDF or an 11 MB image is rejected with the documented message
7.7	Assigning an installer creates a <code>job_assigned</code> notification and (if FCM is configured) a device push
7.8	A job created without an installer shows Unassigned and has no <code>ASSIGNED</code> forcing

08 — Job Orders & Payments

Routes:

Route	Purpose
<code>/job-orders/software</code>	The Project Job Order list
<code>/job-orders/:jobId</code>	Editor opened from an installation job
<code>/job-orders/order/:joId</code>	Editor opened by job-order id (<code>new</code> for a standalone order)

Roles: `SUPER_ADMIN` , `ADMIN_STAFF` , `LIAISON` , `SALES_STAFF` for the list; the editor additionally admits `DESIGNER` (read-oriented access).

A **Job Order** is the commercial document behind a job: what was sold, what materials went into it, what discount applied, and what has been paid.

The list page

Heading: **Project Job Order**. Four document-type tabs, each with a live count:

Tab	docType
Job Order	<code>JOB_ORDER</code>
Quotation	<code>QUOTATION</code>
Sales Invoice	<code>INVOICE</code>
Official Receipt	<code>RECEIPT</code>

Rows with no `docType` are treated as `JOB_ORDER` .

Columns: **JO No.** · **Client** · **Product** · **Type** · **Sale Price** · **Total** · **Status** · **Schedule** · **Created At**. Paginated. Empty tab shows "*No {tab label} records match.*"; load failure shows "**Failed to load job orders.**"

Creating

The primary button changes with the active tab:

Active tab	Button	Behaviour
Quotation	Create Quotation	Goes straight to <code>/job-orders/order/new?doc=QUOTATION</code> — no installation job needed
Any other	Create Project JO	Opens Select Installation Job , listing installations that don't already have a job order

If every installation already has an order: "*No pending installations available for new orders.*"

Expected: a job that already owns a job order never appears in the picker (one job ↔ at most one job order — `job_orders.job_id` is unique).

The editor — a 3-step wizard

Step indicator at the top:

#	Step	Enabled
1	Client & Project	always

#	Step	Enabled
2	Materials / Package	always
3	Payments	only once the order has been saved (tooltip: "Save the job order to record payments")

If the order disappears while you are on step 3, the wizard falls back to step 2.

Step 1 — Client & Project

- **Client** — pick an existing client, or create one inline (Business name, Owner name, Contact no., Email, Address, Client type, auto `CLT-` code). With no clients yet: **"No clients yet."**
- **Project Type** — one of:

Type	Extra fields	Labor incentive formula
System / Software (<code>SOFTWARE</code>)	Software product (required)	none
CCTV Installation (<code>CCTV</code>)	No. of Cameras, rate per camera	<code>cameraCount × cameraRate</code>
Signage Installation (<code>SIGNAGE</code>)	Labor % (default 20)	<code>salePrice × laborPct / 100</code>

- **Price** — the sale price of the system/service.
- **Schedule date** and **Installer (optional)** appear for standalone orders that will become jobs.
- **Remarks / Notes.**

Save is blocked (**Save** disabled) until a client is chosen, and — for `SOFTWARE` type — a product is chosen.

Step 2 — Materials / Hardware Package

A line-item table: **Item name · Description · Qty · Unit Price**, each row optionally linked to an **inventory item** (which drives stock deduction — see [09 — Inventory](#)).

Validation: `quantity` ≥ 1 (integer), `unitPrice` ≥ 0.

Order Summary

Line	Formula
Materials Total	<code>Σ (quantity × unitPrice)</code>
Subtotal	<code>salePrice + materialsTotal</code>
Discount	<code>FIXED</code> : the entered amount · <code>PERCENTAGE</code> : <code>subtotal × discount / 100</code> (shown as <code>-Px (n%)</code>)
Grand Total	<code>max(0, subtotal - discountAmount)</code>
Installer Labor	The labor incentive above — internal cost, never part of the Grand Total

The backend recomputes the grand total with the *same* formula (`job-order-pricing.util.ts`) so the printed invoice and the payment balance can never disagree.

Expected: a discount larger than the subtotal clamps the Grand Total to ₹0, never negative.

Step 3 — Payments

Requires a saved order. See [Payments](#) below.

Saving — `POST /api/job-orders` (upsert)

One endpoint handles create and update. It resolves the target as:

1. `dto.id` if supplied (required to re-save a standalone order) — unknown id → `404 Job order {id} not found`
2. else the existing order for `dto.jobId`
3. else create new

On save, inside **one transaction**:

1. All existing line items are deleted and recreated from the payload.
2. Inventory stock is reconciled for the completed-state change.
3. `ensureLaborEarning` guarantees the installer's labor earning for CCTV/Signage.

Labor earning rules (CCTV / Signage only)

Triggered when the order's status is `FINALIZED`, `ON_GOING` or `COMPLETED`:

Condition	Result
No linked job, or the job has no installer	<code>400 Assign an installer to the job before finalizing.</code>
CCTV with labor ≤ 0	<code>400 Enter the number of cameras and rate per camera before finalizing.</code>
Signage with labor ≤ 0	<code>400 Signage labor is zero – check the total price and labor % before finalizing.</code>
An <code>INSTALLATION</code> earning already exists for that job	No new earning (idempotent)
Otherwise	Creates a <code>PENDING</code> <code>INSTALLATION</code> earning for the installer, amount = labor incentive

Because this runs inside the upsert transaction, a validation failure **rolls the whole save back** — nothing is persisted.

Expected: finalizing a CCTV order twice produces exactly **one** installation earning. `SOFTWARE` orders never create a labor earning.

Document types & printing

The same record prints under four letterheads. The **Move to...** dialog changes `docType` ("Moves this record to the selected tab on the Project JO list and sets the print letterhead.") and immediately re-saves the order.

Doc type	Printed subtitle	PDF filename prefix
<code>JOB_ORDER</code>	Job Order / Delivery Receipt	<code>JO-</code>
<code>QUOTATION</code>	Quotation / Price Estimate	<code>QUO-</code>
<code>INVOICE</code>	Sales Invoice	<code>INV-</code>
<code>RECEIPT</code>	Official Receipt	<code>OR-</code>

Filename pattern: `{prefix}-{first 8 chars of the order id, uppercased}.pdf` (`NEW` when unsaved).

The print template pulls the letterhead from **Company Profile** (name, logo, address, phone, email, website, TIN). The signature block reads **"Conforme / Client Representative"** on a Quotation and **"Received by / Client Representative"** on every other type.

Expected: the print output shows the same Grand Total as the screen and as `GET /api/job-orders/:id/payments`.

Converting a quotation

`POST /api/job-orders/:id/convert` — roles `SUPER_ADMIN`, `ADMIN_STAFF`, `LIAISON`, `SALES_STAFF`.

Body: `scheduleDate` (required, ISO date) and `installerId` (optional).

Inside a transaction it creates the installation **Job**, links it to the order (`jobId`) and forces `docType` back to `JOB_ORDER`.

Case	Result
Order already has a <code>jobId</code>	<code>400 This order is already linked to an installation job.</code>
Unknown order	<code>404 Job order {id} not found</code>
UI failure fallback	"Could not convert this order. Try again."

Expected: after conversion the record moves from the Quotation tab to the Job Order tab, and the new job appears on `/jobs`.

Payments

Operation	Endpoint	Roles
Record	<code>POST /api/job-orders/:id/payments</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code> , <code>SALES_STAFF</code>
List + totals	<code>GET /api/job-orders/:id/payments</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code> , <code>SALES_STAFF</code>
Void	<code>POST /api/payments/:id/void</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>

Recording

Field	Required	Rules
Amount	✓	Number $\geq$ 0.01
Method	✓	<code>CASH</code> · <code>BANK_TRANSFER</code> · <code>GCASH</code> · <code>CHECK</code>
Reference no.		Optional
Proof photo		Optional image URL
Paid at	✓	ISO date string

The recorder is stamped as `recordedById`.

 **Overpayment is not blocked.** There is no server check that `amount ≤ balance`; recording more than the balance produces a **negative balance**. Confirm the intended behaviour before filing this as a bug — it is listed in [20 — Known Gaps](#).

Totals returned by `GET /api/job-orders/:id/payments`

```
grandTotal = computeGrandTotal(salePrice, discount, discountType, items)
totalPaid  = Σ amount of NON-VOIDED payments
balance    = grandTotal - totalPaid
```

Voiding

Roles `SUPER_ADMIN` / `ADMIN_STAFF`. The **Void** button is hidden for everyone else.

Case	Result
Reason shorter than 3 characters	<code>400</code> (validation)
Payment already voided	<code>400 Payment is already voided</code>
Success	Stamps <code>voidedAt</code> , <code>voidReason</code> , <code>voidedById</code> ; the payment stays in the list but stops counting

Expected: voiding a payment increases the balance by exactly that amount and the row remains visible, marked as voided.

Job order statuses

DRAFT → FINALIZED → ON_GOING → COMPLETED , plus CANCELLED .

The API does **not** enforce a transition order — any status can be set on save. What each status *does*:

Status	Effect
DRAFT	No labor earning, no stock deduction
FINALIZED	Triggers the labor earning (CCTV/Signage); counts as revenue in the financial summary and revenue trend
ON_GOING	Labor earning guaranteed
COMPLETED	Labor earning guaranteed + inventory stock is deducted
CANCELLED	Excluded from the outstanding-balances report; reverting from COMPLETED restores stock

Expected: moving a COMPLETED order back to ON_GOING restores the deducted stock (a JOB_ORDER_RESTORE movement appears in the ledger).

Expected behaviour summary

#	Assertion
8.1	Step 3 is disabled until the order is saved
8.2	Grand Total = $\max(0, \text{salePrice} + \sum(\text{qty} \times \text{price}) - \text{discount})$ and matches the API, the print output and the payments panel
8.3	Percentage discount applies to the subtotal (sale price + materials), not the sale price alone
8.4	Installer Labor is displayed but never included in the Grand Total
8.5	Finalizing a CCTV/Signage order without an assigned installer returns the documented 400 and saves nothing
8.6	Repeated saves never duplicate the installation earning
8.7	Saving replaces line items wholesale — removed rows disappear
8.8	Converting a quotation twice returns 400 This order is already linked to an installation job.
8.9	Voided payments are excluded from totalPaid
8.10	Move to... re-saves the record and moves it between list tabs
8.11	Only SUPER_ADMIN / ADMIN_STAFF see the Void action

09 — Inventory

Location: **Settings** → **Inventory Management** (`/settings` , Super Admin) — the same component is also reachable as a standalone page.

Inventory holds the materials/hardware catalogue used as job-order line items, with a full stock ledger.

Item fields

Field	Required	Rules
Name	✓	
Description		Trimmed; empty → <code>NULL</code>
Barcode		Unique when set. Trimmed; empty → <code>NULL</code> . Duplicate → <code>400 That barcode is already used by another item</code>
Unit Price		<code>Decimal(12,2)</code> , default 0
Initial Stock / Stock		Integer, default 0
Low-stock alert		Integer threshold, default 0
Sort order		Integer, default 0
Active		Default <code>true</code>

Listing order: `sortOrder` ascending, then `name` ascending.

By default the list returns **active items only**; pass the include-inactive flag to see deactivated ones.

Barcode lookup

`GET /api/inventory/barcode/:code` returns the matching **active** item. No match → `404 No active inventory item with that barcode` .

Expected: an inactive item is never returned by barcode lookup, even with an exact match.

Manual stock adjustment

`POST /api/inventory/:id/adjust` (roles `SUPER_ADMIN` , `ADMIN_STAFF`).

- Positive `delta` adds stock, negative removes it.
- Adjusting below zero is **rejected**: `400 Stock cannot go below zero` .
- Records a `MANUAL_ADJUST` movement with the note *"Manual restock/adjust"* and the acting user.

The stock ledger

Every stock change writes a `stock_movements` row:

Column	Meaning
<code>delta</code>	Signed change (+ adds, – removes)
<code>balance</code>	Resulting <code>stockQty</code> after this movement
<code>reason</code>	<code>MANUAL_ADJUST</code> · <code>JOB_ORDER_DEDUCTION</code> · <code>JOB_ORDER_RESTORE</code>
<code>jobOrderId</code>	Set for job-order-driven movements

Column	Meaning
userId	Who caused it
note	Free text

GET /api/inventory/:id/movements returns the newest **50** by default. UI columns: **When · Change · Balance · Reason**. Empty: **"No stock movements yet."**

Automatic deduction from job orders

Consumption counts **only while the job order is COMPLETED**. One reconciliation formula covers every case:

```
consumedOld = items on the saved order, if it WAS completed
consumedNew = items on the incoming order, if it IS completed
stockDelta = -(consumedNew - consumedOld) per inventory item
```

Transition	Result
Not completed → COMPLETED	Stock deducted (JOB_ORDER_DEDUCTION)
COMPLETED → any other status	Stock restored (JOB_ORDER_RESTORE)
Editing quantities on an already-completed order	Only the net difference moves
Line item with no inventoryItemId	Ignored entirely

Two deliberate design choices QA must not report as bugs:

1. **Stock is allowed to go negative** through job-order completion — completing a job never blocks on insufficient stock; the shortfall simply shows as a negative/red balance. (Only *manual* adjustment refuses to go below zero.)
2. If an inventory item was deleted, the adjustment is silently skipped rather than erroring.

Expected: completing an order for 5 units of an item with 2 in stock succeeds and leaves a balance of −3 with a JOB_ORDER_DEDUCTION movement.

Low-stock indicator

Items whose stockQty is at or below lowStockAlert are flagged **Low stock** in the list.

Permissions

Operation	Endpoint	Roles
List	GET /api/inventory	any authenticated user
Barcode lookup	GET /api/inventory/barcode/:code	any authenticated user
Movements	GET /api/inventory/:id/movements	any authenticated user
Create	POST /api/inventory	SUPER_ADMIN , ADMIN_STAFF
Update	PATCH /api/inventory/:id	SUPER_ADMIN , ADMIN_STAFF
Adjust stock	POST /api/inventory/:id/adjust	SUPER_ADMIN , ADMIN_STAFF
Delete	DELETE /api/inventory/:id	SUPER_ADMIN , ADMIN_STAFF

Deleting an item sets inventoryItemId to NULL on any job-order line that referenced it (onDelete: SetNull) and cascades its stock movements away.

Expected: deleting an item does **not** delete the job-order lines that used it — those lines keep their name, quantity and price and simply lose the stock link.

Expected behaviour summary

#	Assertion
9.1	Duplicate barcode is rejected with the documented message
9.2	Manual adjustment below zero is rejected; job-order deduction below zero is allowed
9.3	Every stock change writes exactly one ledger row with the correct running balance
9.4	Completing → un-completing a job order nets to zero stock change
9.5	Editing quantities on a completed order moves only the difference
9.6	Barcode lookup ignores inactive items
9.7	Deleting an item nulls the link on existing job-order lines without deleting them

10 — Earnings & Withdrawals

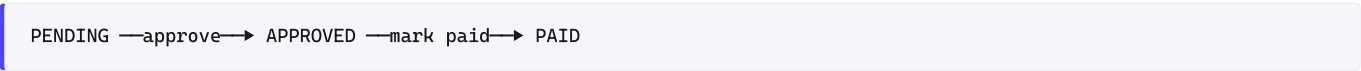
Routes: `/earnings` and `/withdrawals` — visible to **all seven roles**.

Part A — Earnings

An **Earning** is a money credit to a staff member.

Type	Created by
INSTALLATION	Automatically when a CCTV/Signage job order reaches <code>FINALIZED</code> / <code>ON_GOING</code> / <code>COMPLETED</code> — see 08
BONUS	Automatically mirrored from a monthly KPI incentive — see 11
ACTIVATION	Manual allocation
COMMISSION	Manual allocation

Status flow — strictly forward



Any other transition returns `400 Cannot mark a {current} earning as {target}.`

Expected: `PENDING → PAID` (skipping approval) is rejected; `APPROVED → PENDING` is rejected; `PAID` is terminal.

Scoping

`GET /api/earnings` returns:

- **All** earnings for `SUPER_ADMIN` / `ADMIN_STAFF`
- **Only your own** earnings for everyone else

The page

Element	Admin view	Own view
Columns	Team, Recipient, Client, Type, Amount, Status, Recorded, actions	Client, Type, Amount, Status, Recorded
Search placeholder	"Search team member, client, type..."	"Search client, type..."
Actions	Approve, Mark paid	none
Status hints		"Awaiting admin approval", "Released to your account"

A **Cumulative Earnings** figure is shown. Empty: **"No earnings recorded yet."** / **"No matches."**; failure: **"Failed to load earnings."**

Permissions

Operation	Endpoint	Roles
Allocate (create)	<code>POST /api/earnings</code>	<code>SUPER_ADMIN</code>
List	<code>GET /api/earnings</code>	any authenticated user (scoped)
Approve	<code>PATCH /api/earnings/:id/approve</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>
Mark paid	<code>PATCH /api/earnings/:id/paid</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>

Self-action ban: approving or paying **your own** earning returns `403 You cannot approve or pay out your own earning.`

Part B — Withdrawals

A **Withdrawal** is a staff request to cash out.

The balance formula

```
availableBalance = Σ earnings where status ∈ {APPROVED, PAID}
                  - Σ withdrawals where status ∈ {PENDING, APPROVED, RELEASED}
                  (floored at 0)
```

Reasoning behind it:

- **PENDING** earnings are **not** yet spendable.
- **PAID** earnings stay in the earned pool — the **PAID** flag is bookkeeping, it does not remove money.
- **PENDING** and **APPROVED** withdrawals are **reservations** against future payout.
- **RELEASED** withdrawals are permanent deductions.
- **REJECTED** withdrawals return the money to the pool.

Endpoint: `GET /api/withdrawals/balance`.

Expected: the balance never goes negative — it is clamped at 0.

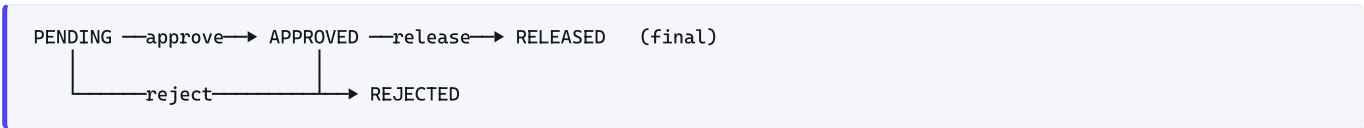
Requesting a withdrawal

`POST /api/withdrawals` — roles **INSTALLER**, **DEVELOPER**, **DESIGNER**, **LIAISON**, **SALES_STAFF**, **ADMIN_STAFF**. (**SUPER_ADMIN** is **not** in the list.)

Field	Required	Rules
Amount	✓	Must not exceed the available balance
Method	✓	GCASH · MAYA · BANK_TRANSFER
Account name	✓	
Account number	✓	

Exceeding the balance → `400 Insufficient balance. Available: ₱{x}, requested: ₱{y}.`

Status flow



Allowed source states per target:

Target	Allowed from
APPROVED	PENDING
REJECTED	PENDING , APPROVED
RELEASED	APPROVED

Anything else → `400 Cannot mark a {current} withdrawal as {target}.`

REJECTED is reachable from **APPROVED** on purpose, so a mistaken approval can be undone before the money is released. **RELEASED** is terminal.

Releasing

`PATCH /api/withdrawals/:id/release` accepts an optional **proof URL** (a photo of the transfer receipt), stored on the withdrawal. The UI shows it in the **Proof** column, or **"No proof attached"**.

Notifications

Every status change notifies the requester:

Status	Notification body
APPROVED	Your ₦x withdrawal has been approved.
REJECTED	Your ₦x withdrawal was rejected.
RELEASED	Your ₦x withdrawal has been released.

Title is `Withdrawal {status lowercased}`; deep link `/withdrawals`.

Scoping and the page

`GET /api/withdrawals` returns all requests for `SUPER_ADMIN` / `ADMIN_STAFF`, and only your own for everyone else.

Columns (admin): **Requested by** · **Amount** · **Via** · **Account** · **Status** · **Proof** · **actions**. Non-admins lose the *Requested by* and action columns. Empty: **"No withdrawal requests yet."** / **"No matches."**; failure: **"Failed to load withdrawal requests."**

Permissions

Operation	Endpoint	Roles
Balance	<code>GET /api/withdrawals/balance</code>	all except <code>SUPER_ADMIN</code>
Request	<code>POST /api/withdrawals</code>	all except <code>SUPER_ADMIN</code>
List	<code>GET /api/withdrawals</code>	any authenticated user (scoped)
Approve	<code>PATCH /api/withdrawals/:id/approve</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>
Reject	<code>PATCH /api/withdrawals/:id/reject</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>
Release	<code>PATCH /api/withdrawals/:id/release</code>	<code>SUPER_ADMIN</code> , <code>ADMIN_STAFF</code>

Self-action ban: processing **your own** withdrawal returns `403 You cannot process your own withdrawal request.` This applies to an `ADMIN_STAFF` user who requested a withdrawal themselves — a second approver is required.

Worked example (use this as a test scenario)

- Admin allocates a `COMMISSION` earning of ₦5,000 to Installer A → status `PENDING` . **Balance = ₦0.**
- Admin approves it → `APPROVED` . **Balance = ₦5,000.**
- Installer A requests ₦3,000 via GCASH → withdrawal `PENDING` . **Balance = ₦2,000.**
- Installer A tries to request ₦2,500 → `400 Insufficient balance. Available: ₦2,000, requested: ₦2,500.`
- Admin approves the withdrawal → `APPROVED` . **Balance still ₦2,000.**
- Admin rejects it instead of releasing → `REJECTED` . **Balance back to ₦5,000.**
- New request of ₦3,000, approved, then released with a proof photo → `RELEASED` . **Balance = ₦2,000**, and the installer receives a *"has been released"* notification.
- Admin marks the original earning `PAID` . **Balance stays ₦2,000** — `PAID` earnings still count as earned.

Expected behaviour summary

#	Assertion
10.1	<code>PENDING</code> earnings do not raise the available balance
10.2	Marking an earning <code>PAID</code> does not change the balance
10.3	A <code>PENDING</code> withdrawal immediately reserves its amount
10.4	Rejecting a withdrawal returns the amount to the balance
10.5	Balance is clamped at 0, never negative
10.6	Illegal status transitions return the documented 400 messages
10.7	Self-approval of earnings and self-processing of withdrawals both return 403
10.8	Every withdrawal status change produces a notification with the peso amount
10.9	Non-admin users see only their own earnings and withdrawals from the API

11 — KPI & Incentives

Locations:

Screen	Route	Roles
Performance scorecard widget	/ (Dashboard)	every KPI role
Analytics & KPI	/analytics	SUPER_ADMIN
KPI Settings	/settings → KPI Settings tab	SUPER_ADMIN

KPI-eligible roles

INSTALLER, DEVELOPER, DESIGNER, LIAISON, SALES_STAFF, ADMIN_STAFF.

SUPER_ADMIN is not KPI-tracked — it is excluded from team KPI listings and from incentive generation.

Scoring

For each KPI definition:

```
score = min(actual / target, 1) × weight      (0 when target ≤ 0)
totalScore = Σ score over all of the role's KPIs
```

Actual values are rounded to 1 decimal for display; scores to 2 decimals.

Incentive tiers

```
totalScore ≥ 95 → 100 % of base bonus
totalScore ≥ 90 → 75 %
totalScore ≥ 85 → 50 %
totalScore ≥ 80 → 25 %
totalScore ≥ 75 → 10 %
below 75       → 0 %
```

incentiveEstimate = baseBonus × tier%

Base bonus is per-user (users.base_bonus , default ₱10,000), editable on the Users screen. It represents the salary grade the bonus is computed from.

Expected: a total score of exactly 95.00 pays 100 %; 94.99 pays 75 %.

Default KPI definitions

Seeded into kpi_definitions once per role at startup, then the **database is the source of truth**. auto: true means the value is computed by the system; auto: false means it must be entered manually.

INSTALLER

KPI	Weight	Target	Unit	Auto
Installation Completion Rate	35	90	%	✓
Proof Submission Rate	30	100	%	✓
Monthly Activity	15	100	%	✓

KPI	Weight	Target	Unit	Auto
Customer Satisfaction	15	90	/100	
Safety Compliance	5	100	%	

Auto formulas (for jobs whose `scheduleDate` falls in the selected month):

```

Installation Completion Rate = completed / total × 100
Proof Submission Rate       = (jobs in WAITING_ACTIVATION|COMPLETED with a proof)
                             / (jobs in WAITING_ACTIVATION|COMPLETED) × 100
Monthly Activity            = min(total / 10, 1) × 100

```

All three return **0** when their denominator is 0.

DEVELOPER

KPI	Weight	Target	Unit	Auto
License Activations	40	10	count	✓
Activation Quality	30	90	%	✓
On-Time Activation Rate	20	95	%	
Quality Score	10	90	/100	

```

License Activations = licenses activated by this developer in the month
Activation Quality  = (their licenses currently ACTIVATED) / (all licenses they ever activated) × 100

```

Note that *Activation Quality* is **all-time**, not month-scoped — a license that later expires lowers it.

DESIGNER

KPI	Weight	Target	Unit	Auto
On-Time Design Completion	35	90	%	✓
Monthly Activity	25	100	%	✓
First Approval Rate	20	80	%	
Design Quality Score	20	90	/100	

Designer actuals do **not** come from a local formula — they are populated from TMS Pro (see below). Until a sync runs, auto designer KPIs read 0.

LIAISON

KPI	Weight	Target	Unit
Project Completion Rate	30	95	%
On-Time Permit Processing	20	100	%
Client Satisfaction	20	90	/100
Documentation Accuracy	15	98	%
Response Time	15	90	/100

All manual.

SALES_STAFF

KPI	Weight	Target	Unit
Sales Target Achievement	35	100	%
Job Orders Closed	25	10	count
New Client Acquisitions	20	5	count
Client Satisfaction	20	90	/100

All manual.

ADMIN_STAFF

KPI	Weight	Target	Unit
Task Completion Rate	30	95	%
Documentation Accuracy	25	98	%
Attendance & Punctuality	25	100	%
Supervisor Rating	20	90	/100

All manual.

Manual override

A stored `KpiResult` with `isManual = true` **always overrides** the auto computation for that KPI, user, month and year.

`POST /api/kpis/manual` (roles `SUPER_ADMIN`, `ADMIN_STAFF`) — from the Analytics page's **Enter Manual KPI** dialog. It upserts on `(userId, month, year, kpiName)` and recomputes the score immediately. Unknown KPI name for that user's role → `404 KPI "{name}" not found for role {ROLE}`.

Expected: entering a manual value for an `auto: true` KPI (e.g. an installer's *Installation Completion Rate*) replaces the computed value and the row shows as manual.

Managing KPI definitions

Settings → **KPI Settings**. Columns: **KPI Name · Weight (%) · Target · Unit · Type**, plus a **Total weight** readout.

Operation	Endpoint	Rules
List for a role	<code>GET /api/kpis/definitions/:role</code>	Seeds defaults on first access
Create	<code>POST /api/kpis/definitions</code>	Always created with <code>auto: false</code> , <code>isCustom: true</code> . Non-KPI role → <code>400 Role {X} does not have KPI tracking</code> . Duplicate name → <code>409 KPI "{name}" already exists for {ROLE}</code>
Update	<code>PATCH /api/kpis/definitions/:id</code>	Renaming a system-tracked (<code>auto: true</code>) KPI → <code>400 Cannot rename a system-tracked KPI</code> . Weight/target/unit are always editable
Delete	<code>DELETE /api/kpis/definitions/:id</code>	Allowed for built-in KPIs too

Roles for all four: `SUPER_ADMIN`, `ADMIN_STAFF`.

Total weight is **not validated to 100**. A role whose weights sum to 120 can score above 100 and always land in the top incentive tier. Verify against product intent — noted in [20 — Known Gaps](#).

Generating incentives

POST /api/kpis/incentives/generate with { month, year, baseBonus? } (roles SUPER_ADMIN, ADMIN_STAFF).

For every **active** user in a KPI role:

1. `base = user.baseBonus || dto.baseBonus || 10000`
2. Compute `totalScore` for the month
3. `bonusAmount = base × incentivePct(totalScore)`
4. Upsert the `Incentive` on (userId, month, year) — status `PENDING` on create, amounts refreshed on re-run
5. **Mirror it into the earnings ledger**

The supplied `dto.baseBonus` is only a **fallback** for users with no base bonus set — it does not override per-user values.

Incentive ↔ Earning mirroring

Linked 1:1 via `Earning.incentiveId`, so repeated generation is idempotent.

Incentive status	Earning status
PENDING	PENDING
APPROVED	APPROVED
PAID	PAID

The earning has `type = BONUS`. If the bonus amount is **0**, any existing linked earning is **deleted** rather than kept at zero.

Approving (PATCH /api/kpis/incentives/:id/approve) or paying (PATCH /api/kpis/incentives/:id/pay) an incentive re-syncs the earning.

Expected:

- Approving an incentive raises the recipient's withdrawable balance by the bonus amount (because the mirrored earning becomes `APPROVED`).
- Regenerating the same month twice produces one incentive and one earning.
- A user who scores below 75 gets a ₱0 incentive and **no** bonus earning.

Viewing incentives

Endpoint	Roles	Returns
GET /api/kpis/incentives?month=&year=	SUPER_ADMIN, ADMIN_STAFF	All, sorted year ↓, month ↓, amount ↓
GET /api/kpis/incentives/mine	every role except SUPER_ADMIN	Your own, newest first

Team KPI (Analytics page)

Heading **Analytics & KPI**. Sections:

- **Total Revenue** (*All-time finalized sales*), **This Month, Last Month** (*Previous month total*)
- **Team KPI Performance** table: Name · Role · Score · Base Bonus · Incentive · Status · Actions, sorted by total score descending. Empty: **"No active team members found."**
- **Enter Manual KPI** dialog. Success toast: **"Saved successfully."**

GET /api/kpis/team?month=&year= powers it (roles SUPER_ADMIN, ADMIN_STAFF); only **active** users in KPI roles are included.

Financial figures on this page

Endpoint	Definition
GET /api/kpis/financial-summary	Revenue = $\sum \text{salePrice}$ of FINALIZED job orders. Growth = $(\text{current} - \text{previous}) / \text{previous} \times 100$; when the previous month is 0 $\rightarrow 100\%$ if this month is positive, else 0 %. Also returns revenue split by product name (Other when no product).
GET /api/kpis/revenue-trend	Last 6 months including the current one, oldest first, again FINALIZED only

⚠ Revenue here counts **only** **FINALIZED** orders — an **ON_GOING** or **COMPLETED** order is *excluded*. This is a real behaviour to confirm with the product owner; see [20 — Known Gaps](#).

Designer KPI — TMS Pro integration

Designer points come from an external system (TMS Pro).

Endpoint	Purpose
GET /api/kpis/designers/points	Read-only preview: matched designers + the raw TMS roster
POST /api/kpis/designers/sync	Pull points and write them as manual KPI results

Configuration: `TMS_KPI_API_URL` (default `https://tmspro.up.railway.app`) and `TMS_KPI_API_TOKEN` (needs the `kpi:read` ability). The call is `GET {base}/api/v1/kpi/export?from=&to=`.

Matching: designers (primary role or additional role `DESIGNER`, active only) are matched to TMS employees **by email first**, then by normalised full name. The UI's **TMS Match** column shows how each matched, or that it didn't.

Sync maths: each designer's `total_points` is distributed across their `DESIGNER` KPI definitions **proportionally to weight**:

$$\text{allocated}(kpi) = \text{totalPoints} \times (kpi.\text{weight} / \sum \text{weights})$$

and stored as a manual `KpiResult`. Default date range when none is given: the first to the last day of the requested month.

Error handling (all surface as **400** with a readable reason):

Situation	Message
No token configured	<code>TMS_KPI_API_TOKEN</code> is not configured on the server.
Network failure	Could not reach the TMS KPI API.
HTTP 401	TMS KPI API rejected the token (401 Unauthorized).
Other non-2xx	TMS KPI API returned HTTP {status}.
Non-JSON body (HTML login redirect)	TMS KPI API did not return JSON — the token may be invalid or lack the <code>kpi:read</code> ability.

Unmatched designers are returned with `applied: false` and `totalScore: 0` — they are **not** written to.

Expected behaviour summary

#	Assertion
11.1	<code>score = min(actual/target, 1) × weight</code> — exceeding the target never earns more than the full weight
11.2	Tier boundaries are inclusive at 95 / 90 / 85 / 80 / 75
11.3	Manual values override auto values for the same KPI/month
11.4	KPI definitions seed once per role; edits persist and are not re-seeded

#	Assertion
11.5	System-tracked KPIs cannot be renamed
11.6	Incentive generation is idempotent per (user, month, year)
11.7	Incentive status and the mirrored <code>BONUS</code> earning status always match
11.8	A <code>P0</code> incentive leaves no bonus earning behind
11.9	<code>SUPER_ADMIN</code> never appears in team KPI or incentive generation
11.10	TMS sync distributes points by weight and writes nothing for unmatched designers
11.11	Every TMS/Finara failure mode returns a readable 400, never a 500

12 — Dev Projects

Route: `/dev-projects` · Roles: `DEVELOPER` , `ADMIN_STAFF` , `SUPER_ADMIN`

Tracks internal software development: a per-project **timer**, a manual **progress %**, and **reports** that a developer submits for admin review.

Project fields

Field	Required	Notes
Software name	✓	
Description		Optional
Developer	✓	Must be a user whose primary role is <code>DEVELOPER</code> → otherwise <code>404 Developer not found</code>
Estimated timeframe (hours)		<code>targetHours</code>
Project start date		
Deadline / end date		
Status	auto	<code>NOT_STARTED</code> → <code>IN_PROGRESS</code> → <code>PENDING</code> → <code>COMPLETED</code>
Progress	manual	0–100 %

Ownership rules

- A `DEVELOPER` creating a project always becomes its own developer — the supplied `developerId` is ignored.
- A `SUPER_ADMIN` creating a project **must** supply `developerId` → otherwise `403 developerId is required`.

Visibility

Role	Sees
<code>DEVELOPER</code>	Only their own projects
<code>ADMIN_STAFF</code> , <code>SUPER_ADMIN</code>	All projects

Accessing someone else's project → `403 You do not have access to this development project`.

Note the asymmetry, which QA should verify explicitly:

- **Viewing** a project is allowed for `SUPER_ADMIN` and `ADMIN_STAFF`.
- **Acting** on a project (start/pause/resume/stop/progress/report) is allowed for the owning developer and `SUPER_ADMIN` only — `ADMIN_STAFF` gets 403.
- **Editing** project metadata (`PATCH /api/dev-projects/:id`) is `SUPER_ADMIN` only.

The timer

Buttons: **Start**, **Pause/Resume**, **Stop**. A floating widget in the app shell shows the running project.

Status semantics

Status	<code>startedAt</code>	Meaning
<code>NOT_STARTED</code>	null	Never started
<code>IN_PROGRESS</code>	a timestamp	Running

Status	startedAt	Meaning
IN_PROGRESS	null	Paused — the widget stays visible
PENDING	null	Stopped, work not finished
COMPLETED	null	Progress reached 100 %

Time accounting

Two counters exist so paused time is never double-counted:

- `runSeconds` — seconds banked by pauses **within the current run**; reset to 0 on start and on any run-ending transition.
- `totalMinutes` — grows **once**, when the run ends, by `round((runSeconds + lastSegmentSeconds) / 60)`.

Each start/resume opens a `DevProjectSession` row; each pause/stop closes it with its `endedAt` and rounded `minutes`.

Transition rules

Action	Precondition	Error when violated
Start	Not already <code>IN_PROGRESS</code> , not <code>COMPLETED</code>	403 This project is already being worked on / 403 This project is already completed
Pause	<code>IN_PROGRESS</code> and running (<code>startedAt</code> set)	403 This project is not currently running
Resume	<code>IN_PROGRESS</code> and paused (<code>startedAt</code> null)	403 This project is not currently paused
Stop	<code>IN_PROGRESS</code> (running or paused)	403 This project is not currently running

One active timer per developer

Starting project B while project A is running **auto-stops A first**, inside the same transaction: A's open session is closed, its time is banked into `totalMinutes`, and A becomes `PENDING`.

Expected: a developer can never have two projects in the running state at once, and no time is lost during the hand-over.

Progress

PATCH `/api/dev-projects/:id/progress` with `{ progressPercent }`.

Value	Effect
≥ 100	Status → <code>COMPLETED</code> ; ends the run (banks the open segment + paused seconds into <code>totalMinutes</code> , clears <code>runSeconds</code> and <code>startedAt</code>)
< 100 on a <code>COMPLETED</code> project	Status → <code>PENDING</code> (re-opened)
< 100 otherwise	Status unchanged

Expected: setting a running project to 100 % stops the timer and records the elapsed time — no manual Stop needed.

Reports

A developer posts a report on their own project (`POST /api/dev-projects/:id/reports`).

Field	Required	Notes
Title	✓	
Comment		Optional

Field	Required	Notes
Checklist	✓	Array of { label, done }
Tag admin to review		Must be a SUPER_ADMIN or ADMIN_STAFF user → otherwise 404 Reviewer not found

Report status: PENDING → REVIEWED .

Tagging an admin sends them a "New dev report to review" notification: *A report "{title}" on "{project}" was tagged for your review.*

UI: **New report** dialog with Title, Comment (optional), Checklist, *Tag admin to review (optional)* (**No tag** is the default). Empty state: **"No reports yet."**; failure: **"Failed to post report. Try again."**

Feedback

POST /api/dev-projects/reports/:reportId/feedback — roles SUPER_ADMIN , ADMIN_STAFF .

- A non-Super-Admin who was **not** tagged gets 403 You were not tagged on this report . A SUPER_ADMIN may reply to any report.
- Posting feedback flips the report to REVIEWED in the same transaction.
- The report author is notified ("**Feedback on your report**") — unless they are the one leaving the feedback.

Expected: feedback and the status change are atomic — a failure leaves the report PENDING with no feedback row.

The page

Per project: **Software · Developer · Status · Progress · Time / Target · Project Timeframe · Started / Ended / Duration**, with **Start / Stop, View Progress, Session History** and **Reports**.

Empty: **"No software development projects yet."**; failure: **"Failed to load development projects."** / **"Failed to create project. Try again."**

Supporting endpoints

Endpoint	Returns	Roles
GET /api/dev-projects/developers	Active users with primary role DEVELOPER , by name	DEVELOPER , SUPER_ADMIN , ADMIN_STAFF
GET /api/dev-projects/reviewers	Active SUPER_ADMIN + ADMIN_STAFF , by name	same
GET /api/dev-projects/active	The caller's currently IN_PROGRESS project (for the floating widget)	same

Expected behaviour summary

#	Assertion
12.1	A developer sees and can act on only their own projects
12.2	ADMIN_STAFF can view all projects but cannot start/stop/edit them
12.3	Starting a second project auto-stops the first and banks its time
12.4	Pause → Resume → Stop totals the same minutes as an uninterrupted run of equal length
12.5	Paused time is never counted twice in totalMinutes
12.6	Progress 100 % completes the project and ends the run
12.7	Dropping a completed project below 100 % re-opens it as PENDING
12.8	Tagging a non-admin as reviewer returns 404

#	Assertion
12.9	Feedback sets the report to <code>REVIEWED</code> and notifies the author
12.10	A user with primary role other than <code>DEVELOPER</code> cannot be assigned a project

13 — Financial Reports & Analytics

Route: `/financial-reports` · Roles: `SUPER_ADMIN` , `ADMIN_STAFF` , `SALES_STAFF`

All four endpoints live under `/api/reports/financial` and share the same role guard applied at the controller level.

Everything here is driven by **payments**, not by invoices — a job order only appears once money is expected or received.

Collections summary

GET `/api/reports/financial/collections?from=&to=`

Field	Definition
<code>totalCollected</code>	Σ amount of non-voided payments inside the date filter
<code>byMethod[]</code>	<code>{ method, total, count }</code> per payment method, filtered
<code>byMonth[]</code>	Last 6 calendar months including the current one , oldest first, <code>YYYY-MM</code> keys — ignores the from/to filter
<code>recentPayments[]</code>	The 10 newest non-voided payments inside the filter, with <code>clientName</code>

Date filtering:

- `from` → `paidAt >= new Date(from)`
- `to` → `paidAt <= new Date(to + 'T23:59:59.999Z')` — i.e. the **whole** end day, interpreted in **UTC**

Expected: a payment made at 23:30 local time on the `to` date is included or excluded consistently with the UTC boundary — verify against your timezone before reporting an off-by-one-day bug.

Expected: the 6-month trend bars stay the same when you narrow the from/to range; only the totals, method breakdown and recent list change.

Outstanding balances

GET `/api/reports/financial/outstanding`

Walks every job order **except** `CANCELLED` and returns those with a balance:

```
grandTotal = salePrice +  $\Sigma$ (qty × unitPrice) - discount    (clamped at 0)
totalPaid  =  $\Sigma$  non-voided payments
balance    = grandTotal - totalPaid
```

Rows with `balance <= 0.005` are dropped (a floating-point tolerance, so a fully paid order never lingers because of rounding).

Columns: **Job Order** · **Client** · **Grand Total** · **Paid** · **Balance** · **Last Payment**. Empty: **"No payments recorded."**

Expected: `DRAFT` orders **do** appear in outstanding (only `CANCELLED` is excluded). A fully paid order disappears from the list.

Client payment history

GET `/api/reports/financial/client/:clientId`

Returns `{ clientId, clientName, payments[] }` — all non-voided payments across **all** of that client's job orders, newest first, each carrying its `jobOrderId`. An unknown client id returns `clientName: "Unknown"` with an empty list rather than a 404.

Empty state: **"No payments for this client."**

CSV export

GET /api/reports/financial/export

Export	Columns
Collections	method, total, count
Outstanding	jobOrderId, client, grandTotal, totalPaid, balance, lastPaymentAt (empty string when never paid)

Expected: the CSV row count matches the on-screen table row count for the same filter.

The page

Heading **Financial Reports**, with a date-range filter, the summary cards, the method breakdown, the 6-month trend, the outstanding table, the client history drill-down, and a **Share** action.

Analytics page (Super Admin)

Route: /analytics . Covered in [11 — KPI & Incentives](#), but the revenue figures deserve a separate call-out here because they are computed differently from the financial reports:

Screen	Revenue basis
Financial Reports	Actual payments received (non-voided)
Analytics / Dashboard	Σ salePrice of FINALIZED job orders — <i>not</i> payments, and excluding ON_GOING and COMPLETED orders

These two numbers are expected to differ. What QA should confirm with the product owner is whether the Analytics figure should really exclude ON_GOING / COMPLETED orders — see [20 — Known Gaps](#).

Also note the Analytics revenue uses salePrice only — it **ignores materials and discounts**, whereas the financial reports use the full grand total.

Expected behaviour summary

#	Assertion
13.1	Voided payments are excluded from every figure on this page
13.2	The to filter includes the whole end day
13.3	The 6-month trend is unaffected by the from/to filter
13.4	recentPayments is capped at 10
13.5	CANCELLED job orders never appear in outstanding balances
13.6	An order paid to the cent (balance $\leq$ ₱0.005) drops off the outstanding list
13.7	CSV export matches the on-screen data for the same filter
13.8	SALES_STAFF can open the page; LIAISON , INSTALLER , DEVELOPER , DESIGNER get 403 from the API

14 — Leads & the Public Landing Page

The landing page

Route: `/` **when signed out**. Signing in replaces it with the dashboard; there is no separate marketing URL.

It is a full marketing page for METRIQA covering the operational story (*Office books the installation job → Installer gets it instantly on mobile → Photos, signature, GPS from the site → License goes live against the proof → Pay posts automatically, withdraw anytime*), plus feature sections for the KPI Engine, Dev Project Timer, Proof of Service and the field app.

There is also a public `/developers` page (no authentication required).

Download / lead capture

The **Get METRIQA** call to action opens a form offering two platforms:

Platform	Value stored
Android APK	<code>ANDROID_APK</code>
Desktop App (installs as a PWA in any modern browser)	<code>DESKTOP_PWA</code>

Form fields and their server-side validation:

Field	Rules
Company / Business Name	2–160 characters
Contact Person	2–120 characters
Contact Number	7–40 characters
Email	Valid email format; stored trimmed and lower-cased
Code	Optional, exactly 6 characters when present
Platform	Must be <code>ANDROID_APK</code> or <code>DESKTOP_PWA</code>

Email verification (OTP)

- Visitor enters their email → `POST /api/download-leads/send-code` (rate limit **5 / 60 s** per IP).
- A **6-digit** code is emailed via **Resend** and held **in memory** on the server.
 - Time to live: **10 minutes**
 - Resend cooldown: **60 seconds** → `400 Please wait a minute before requesting another code.`
 - Maximum **5** verification attempts per code
- The response is `{ sent: true | false }`.

`sent: false` is returned — **not** an error — when:

- `RESEND_API_KEY` is not configured, or
- Resend rejected the send (e.g. the sandbox sender can only reach the account owner's address).

In that case the form proceeds **without** a code and the lead is stored with `emailVerified = false`.

- Submitting the form with a code → `POST /api/download-leads` (rate limit **10 / 60 s**). The code is verified and **consumed**; the lead is stored with `emailVerified = true`.

Verification failures:

Situation	Message
No pending code, or expired	<code>400 Verification code expired – request a new one.</code>

Situation	Message
5 attempts already used	400 Too many attempts – request a new code. (the code is discarded)
Wrong code	400 Incorrect verification code. (attempt counter increments)

Important for QA: codes live in **process memory**. Restarting the API invalidates all pending codes — visitors simply request a new one. Do not report "code stopped working after a deploy" as a defect.

Expected: a successful submission returns `{ id }` and the file download begins from `/downloads/...` with a forced attachment header.

Download Leads (admin)

Route: `/download-leads` · Roles: `SUPER_ADMIN` , `ADMIN_STAFF`

Heading **Download Leads**, with two tabs:

Tab 1 — Downloads

Locally stored landing-page leads, newest first.

Columns: **Company** · **Contact Person** · **Contact No** · **Email** · **Platform** · **Date**. The Platform cell shows a phone icon + *Android APK*, or a monitor icon + *Desktop PWA*.

Empty: "**No leads captured yet.**" / "**No matches.**"; failure: "**Failed to load leads.**"

Endpoint: `GET /api/download-leads` .

Tab 2 — Finara

A **live proxy** to the Finara ERP leads export — nothing is stored locally, and the tab only fetches when you open it.

Columns: **Name** · **Company** · **Email** · **Phone** · **Message** · **Source** · **Status** · **Date**. Statuses: `NEW` , `CONTACTED` , `CLOSED` .

Endpoint: `GET /api/download-leads/finara` → `GET {FINARA_API_URL}/api/leads/export` with an `X-API-Key` header.

Failure modes (all **400** with a readable reason):

Situation	Message
<code>FINARA_API_KEY</code> unset	<code>FINARA_API_KEY</code> is not configured on the server.
Network failure	Could not reach the Finara API.
HTTP 401	Finara API rejected the key (401 Unauthorized).
Other non-2xx	Finara API returned HTTP {status}.
Non-JSON body	Finara API did not return JSON – the API key may be invalid.

Known environment issue: the Finara production deployment has been observed returning **404** for `/api/leads/export` . That surfaces here as `Finara API returned HTTP 404.` — an upstream problem, not a METRIQA defect.

Permissions

Endpoint	Auth
<code>POST /api/download-leads/send-code</code>	Public , 5/min per IP
<code>POST /api/download-leads</code>	Public , 10/min per IP

Endpoint	Auth
GET /api/download-leads	SUPER_ADMIN , ADMIN_STAFF
GET /api/download-leads/finara	SUPER_ADMIN , ADMIN_STAFF

Expected: the two public endpoints work with **no** Authorization header; the two admin endpoints return 401 without one.

Expected behaviour summary

#	Assertion
14.1	Requesting a second code within 60 s returns the cooldown message
14.2	A code expires exactly 10 minutes after it was sent
14.3	The 6th wrong attempt discards the code with <i>Too many attempts</i>
14.4	With Resend unconfigured, <code>send-code</code> returns <code>{ sent: false }</code> and the lead can still be submitted, stored as <code>emailVerified: false</code>
14.5	A submitted code is single-use — reusing it returns <i>expired</i>
14.6	Emails are stored lower-cased and trimmed
14.7	The 6th <code>send-code</code> call inside a minute returns 429
14.8	The Finara tab fetches only when opened and surfaces upstream failures as readable 400s
14.9	Landing page is shown only to signed-out visitors at <code>/</code>

15 — Settings & Administration

Route: `/settings` · Role: `SUPER_ADMIN` only

Six tabs:

Tab	Content
Company Profile	Letterhead details used on printed documents
Users & Roles	Team management (also reachable at <code>/users</code>)
KPI Settings	Per-role KPI definitions + designer TMS sync — see 11
Inventory Management	Item catalogue and stock ledger — see 09
Database Management	Backups, restore, and per-module reset
Audit Logs	System activity trail

Company Profile

Singleton record. Feeds the print letterhead on every job order / quotation / invoice / receipt.

Field	Notes
Business name	
Address	
Phone	
Email	
Website	
TIN / Tax ID	
Company Logo	Uploaded image; stored as a URL

Operation	Endpoint	Roles
Read	<code>GET /api/company-profile</code>	any authenticated user
Update	<code>PATCH /api/company-profile</code>	<code>SUPER_ADMIN</code>

Failure states: **"Failed to load company profile."** / **"Failed to save."**

Expected: changing the business name or logo is reflected on the next job-order print preview without a restart.

Users & Roles

Heading **Team**. Columns: **Name** · **Email** · **Phone** · **Roles** · **Base Bonus** · **Status** · **Edit**.

Creating a user

Field	Required	Rules
Full name	✓	
Email	✓	Valid email, unique → <code>409 A user with this email already exists</code>

Field	Required	Rules
Temporary password	✓	Minimum 8 characters , hashed with bcrypt cost 12
Primary role	✓	One of the 7 roles
Additional Roles		Any subset; the primary role is filtered out automatically
Phone		Optional
Base Bonus		Number ≥ 0, default ₱10,000 — drives KPI incentive amounts

Editing a user

PATCH /api/users/:id (SUPER_ADMIN).

- Changing the email to one already in use → 409 Email already in use by another account
- Changing the **primary role** removes that role from additionalRoles
- Supplying additionalRoles **replaces the whole set** (delete-all then create)
- Omitting phone clears it to NULL

Role management endpoints

Operation	Endpoint
Add an additional role	POST /api/users/:id/roles (idempotent upsert)
Remove an additional role	DELETE /api/users/:id/roles/:role
Change the primary role	PATCH /api/users/:id/primary-role

Activate / deactivate

PATCH /api/users/:id/activate and /deactivate (SUPER_ADMIN).

Expected: a deactivated user cannot log in, **and** an already-signed-in session stops working on the next API call (the JWT strategy re-checks isActive).

Reading users

Endpoint	Roles	Returns
GET /api/users/me	any authenticated user	Your own record
PATCH /api/users/me	any authenticated user	Self-service profile update — see 03
GET /api/users?role=	SUPER_ADMIN , ADMIN_STAFF	All users; the role filter matches primary or additional role
POST /api/users	SUPER_ADMIN	Create

Password hashes are **never** returned — every response uses a safe select (id, email, fullName, role, phone, isActive, mfaEnabled, baseBonus, createdAt, additionalRoles).

Empty: "No team members yet." / "No matches."; failure: "Failed to load the team."

The **MFA** column reflects the mfaEnabled flag, but no OTP challenge exists in the login flow. See 20 — Known Gaps.

Database Management

All endpoints below are SUPER_ADMIN only (guard applied at the controller).

Backups

Stored in `<cwd>/backups`, filename pattern `sdmp-<ISO timestamp>.sql` (colons and dots replaced with `-`). The list shows **File · Size · Created**, newest first. Only files matching `^[\\w.-]+\\.sql$` are listed or served.

Action	Endpoint	Notes
List	GET /api/backups	
Create now	POST /api/backups	Native <code>mysqldump</code> when available, otherwise a pure-JS fallback dumper
Download	GET /api/backups/:filename/download	Forced download
Delete	DELETE /api/backups/:filename	
Upload	POST /api/backups/upload	

Automatic: a full backup runs nightly at **03:00** and prunes everything past the newest 14.

Dump strategy: the service resolves `mysqldump` from `MYSQLDUMP_PATH`, then (on Windows) from `C:\Program Files\MySQL\*bin\mysqldump.exe`, then from `PATH`. If the binary is missing entirely (`ENOENT` — e.g. a Railway container), it falls back to a JavaScript dumper that needs no MySQL client. Any *other* failure deletes the partial file and returns `500 Backup failed: {real reason from stderr}`.

Expected: a backup failure never leaves a zero-byte `.sql` file behind, and the error message carries the actual `mysqldump` `stderr` rather than a generic "Internal server error".

Upload rules:

Rule	Value
Field name	<code>file</code>
Max size	200 MB
Extension	Must end in <code>.sql</code> → otherwise <code>400 Only .sql backup files can be uploaded.</code>
No file at all	<code>400 No file uploaded.</code>
Stored name	<code>restored-{timestamp}-{sanitised base}.sql</code> — non- <code>[\\w.-]</code> characters become <code>_</code> , base truncated to 80 chars, empty base becomes <code>backup</code>

Expected: uploading `my report (final).sql` produces a filename like `restored-1753000000000-my_report__final_.sql`, and path-traversal attempts in the original name cannot escape the backups directory.

Restore

POST /api/backups/:filename/restore

The dialog asks **What to restore** and requires **Confirm with your login password**.

Field	Rules
<code>password</code>	The requesting Super Admin's own login password. Wrong → <code>401 Incorrect password</code>
<code>full</code>	<code>true</code> = whole database; <code>false</code> = selected modules
<code>modules</code>	Module ids, required when <code>full</code> is false

Full restore executes the dump directly against the live database.

Module restore is safer and layout-independent:

- Creates a scratch database `sdmp_restore_{timestamp}`
- Loads the entire dump into it
- With `FOREIGN_KEY_CHECKS=0`, for each selected module table present in the dump: `DROP TABLE` in live → `CREATE TABLE ... LIKE` scratch → `INSERT ... SELECT`

4. Re-enables foreign key checks
5. **Always** drops the scratch database, even on failure

Situation	Result
No modules selected	400 Select at least one module to restore.
Unknown module id	400 Unknown module: {id}
None of the selected modules exist in the dump	400 None of the selected modules had tables in this backup.
SQL failure	500 Restore failed: {reason}

Module → table mapping:

Module id	Tables
jobs	jobs , installation_proofs
job-orders	job_orders , job_order_items
dev-projects	dev_projects , dev_project_sessions , dev_project_reports , dev_project_report_feedback
licenses	licenses
earnings	earnings
withdrawals	withdrawals
kpi	kpi_results , incentives
notifications	notifications
nenpos-clients	nenpos_clients
audit-logs	audit_logs

Reset Data

GET /api/backups/reset/modules lists the resettable modules with a **live row count**; POST /api/backups/reset/:moduleId performs the wipe.

Only **transactional** data is resettable. Master data — Users, Clients, Products, Company Profile, KPI Definitions, Inventory — is deliberately **not** resettable here.

Module	Label	Also does
jobs	Installation Jobs	Unlinks job orders and earnings from the jobs, deletes proofs
job-orders	Job Orders	Deletes line items first
dev-projects	Dev Projects	Deletes feedback → reports → sessions → projects
licenses	Licenses	Unlinks licenses from jobs first
earnings	Earnings	
withdrawals	Withdrawals	
kpi	KPI Results & Incentives	Unlinks incentives from earnings; KPI definitions are kept
notifications	Notifications	
nenpos-clients	Nenpos Clients	
audit-logs	Audit Logs	

Safety sequence, in order:

1. Unknown module id → 400 Unknown module: {id}
2. **Password check** — the Super Admin's own login password, wrong → 401 Incorrect password

3. Row count captured for the result
4. **Automatic backup** — reuses the `backupFilename` the client already downloaded if it still exists, otherwise takes a fresh dump. If the backup fails: `400 Auto-backup failed – reset aborted for safety. {reason}` and **nothing is deleted**
5. The wipe runs inside a **single transaction**, children before parents

The response is `{ module, label, deleted, backup }`.

Expected: every reset is preceded by a real backup file; a reset can never proceed if the backup step fails.

Audit Logs

Route: `/audit-logs` (or Settings → Audit Logs). Columns: **When · User · Action · IP address · Device**. Empty: **"No activity recorded yet."** / **"No matches."**; failure: **"Failed to load audit logs."**

What gets logged

- Every `POST`, `PUT`, `PATCH`, `DELETE`
- `POST /api/auth/login` — **both successes and failures**
- `POST /api/auth/logout`, `POST /api/auth/refresh`

Action names:

Request	Action recorded
Login	User Login , or Failed User Login ({status})
Logout	User Logout
Refresh	Token Refresh
<code>POST /api/clients</code>	Created Client
<code>PATCH /api/licenses/...</code>	Updated License
<code>DELETE /api/users/...</code>	Deleted User

Friendly resource names are mapped for: Client, Product, License, NENPOS Client, Job, Job Order, Dev Project, Earning, Withdrawal, User, KPI, Company Profile, Backup. Unmapped resources fall back to the raw path segment.

Metadata captured

`{ method, url, payload, error?, response? }` where `payload` is the request body with `password`, `refreshToken`, `token`, `secret` and `mfaSecret` replaced by `*****`. Also stored: IP address and user-agent string.

Expected: searching the audit log for a plaintext password returns nothing — credentials must never appear, including on failed logins.

⚠ `GET /api/audit-logs` is guarded as **SUPER_ADMIN only**, while the admin-web route `/audit-logs` also admits **ADMIN_STAFF**. See [20 — Known Gaps](#).

Expected behaviour summary

#	Assertion
15.1	Every <code>/settings</code> tab is unreachable for all roles except <code>SUPER_ADMIN</code>
15.2	Passwords must be ≥ 8 characters at creation
15.3	Duplicate emails are rejected on both create and edit

#	Assertion
15.4	Password hashes never appear in any users API response
15.5	Deactivation takes effect on the next API call, not just the next login
15.6	Reset and restore both require the acting Super Admin's own password
15.7	A reset always produces (or reuses) a backup first, and aborts if the backup fails
15.8	Reset never touches Users, Clients, Products, Company Profile, Inventory or KPI definitions
15.9	Module restore drops its scratch database even when the restore fails
15.10	Backup upload rejects non- <code>.sql</code> files and files over 200 MB
15.11	Audit log payloads have all sensitive keys masked
15.12	Failed logins appear in the audit log with the <code>Failed User Login</code> action

16 — Mobile App (Beulah Field)

React Native / Expo Router app for field staff, talking to the **same** NestJS API as the web portal. Distributed as an Android APK (`mobile/beulah-field-v1.0.0.apk` , also downloadable from the landing page).

Authentication

Screen: **Beulah Field** → *Sign in to your account* → **Sign In**.

- Same `POST /api/auth/login` as the web.
- Tokens are stored in `expo-secure-store` (`beulah_access` , `beulah_refresh`).
- A 401 triggers an automatic access-token refresh and a retry; if the refresh fails the app signs out.
- The auth gate at `/` redirects to `/login` when there is no session, or to the tabs when there is.

Expected: killing and reopening the app keeps you signed in until the refresh token expires (7 days by default).

Backend environment switcher

Profile → **Server** lets you switch between two backends and persists the choice in secure store (`beulah_api_env` , `beulah_local_url`):

Env	URL
prod (default)	<code>EXPO_PUBLIC_API_URL</code> , else <code>app.json</code> → <code>expo.extra.apiUrl</code> (currently the Tailscale Funnel URL), else <code>http://10.0.2.2:3001/api</code>
local	Editable at runtime; defaults to <code>http://192.168.1.246:3001/api</code>

The selection is applied at boot and takes effect immediately when saved.

Expected (QA): switching to a local backend and back to prod requires no reinstall; a wrong local URL produces network errors on that env only.

Emulator/device URL cheat-sheet:

Target	URL
Android emulator	<code>http://10.0.2.2:<port>/api</code>
iOS simulator	<code>http://localhost:<port>/api</code>
Physical device	<code>http://<computer LAN IP>:<port>/api</code>

Tabs

Material top-tab bar. Which tabs appear depends on the role:

Tab	Icon	Shown to
Dashboard		everyone
My Jobs		non-admins (hidden for <code>SUPER_ADMIN</code> / <code>ADMIN_STAFF</code>)
Earnings		non-admins
Menu		admins only (<code>SUPER_ADMIN</code> / <code>ADMIN_STAFF</code>)
Profile		everyone

Dashboard

Greeting: *Welcome, {first name}* 🍌 with the subtitle **System overview** (admin) or **Your overview**.

Admin cards	Field-staff cards
Clients, Active Clients, Licenses, Active Licenses, Open Jobs, Pending Withdrawals	Available Balance, Open Jobs, Completed Jobs, Pending Withdrawals

My Jobs

Jobs assigned to the signed-in installer (`GET /api/jobs` , server-scoped), pull-to-refresh, status badges. Tapping a row opens the job detail.

Job detail — `/job/[id]`

Actions:

1. **Start job** → `PATCH /api/jobs/:id/status` → `ON_GOING`
2. **Proof of Installation:**
 - Capture a photo with the camera
 - Grab GPS coordinates
 - Upload the image → `POST /api/uploads/images`
 - Submit → `POST /api/jobs/:id/proof` → job becomes `WAITING_ACTIVATION`
3. **Mark complete** → blocked until the license is activated, with the same server message as the web

The **Submitted Proof** section shows what was captured.

All the server rules in [07 — Installations](#) apply identically — the mobile app is a different client, not a different ruleset.

Earnings

- **Available Balance** — `GET /api/withdrawals/balance`
- **Earnings** list — empty state "No earnings yet."
- **Withdrawals** list — empty state "No withdrawals yet."
- **Request Withdrawal** form: Method, Account name, Account number, amount. The same insufficient-balance rule applies.

Menu (admins)

A tile grid headed **Management**:

Tile	Route	Restriction
 Clients	<code>/admin/clients</code>	
 Products	<code>/admin/products</code>	
 Licenses	<code>/admin/licenses</code>	
 Job Orders	<code>/admin/job-orders</code>	
 Jobs	<code>/admin/jobs</code>	
 Withdrawals	<code>/admin/withdrawals</code>	
 Users	<code>/admin/users</code>	<code>SUPER_ADMIN</code> only
 Audit Logs	<code>/admin/audit-logs</code>	<code>SUPER_ADMIN</code> only

Tapping a job order opens **Job Order Payment** (`/admin/job-orders/[id]`).

Profile

Account info, the **Server** environment switcher, and **Sign Out**.

Admin section guard

`/admin/*` requires `SUPER_ADMIN`, `ADMIN_STAFF` or `SALES_STAFF`; anyone else is redirected back to the tabs.

Note the mismatch: the **Menu tab** only appears for `SUPER_ADMIN` and `ADMIN_STAFF`, but the `/admin` **route guard** also admits `SALES_STAFF`. A sales user therefore has no way into the admin section from the UI even though the routes would accept them. Logged in [20 — Known Gaps](#).

Push notifications

- The app requests notification permission and registers its FCM token via `POST /api/notifications/device-token`.
- `DELETE /api/notifications/device-token/:token` unregisters it.
- One user can hold several tokens (phone, tablet, web); tokens FCM reports as unregistered are pruned automatically.
- **Expo Go cannot resolve native FCM tokens** — a dev/standalone build (`npx expo prebuild` + EAS build) with `google-services.json` is required.
- Without backend FCM credentials the app still works; only background push is missing.

Expected: with push unavailable, in-app data still refreshes normally through the API — no crash, no blocking error.

Expected behaviour summary

#	Assertion
16.1	Tokens survive an app restart and refresh silently on 401
16.2	Admins do not see <i>My Jobs / Earnings</i> ; field staff do not see <i>Menu</i>
16.3	An installer's job list contains only their own jobs
16.4	Proof submission moves the job to <code>WAITING_ACTIVATION</code> and is visible on the web portal immediately
16.5	<i>Mark complete</i> is blocked with the same message as the web when no license is activated
16.6	The Server switcher persists across restarts and defaults to prod
16.7	<code>SALES_STAFF</code> and below cannot reach <code>/admin/*</code> screens from the UI
16.8	Withdrawal requests obey the same balance rules as the web

17 — Status & Enum Reference

Every enum in the system, its values, and where transitions are enforced.

UserRole

SUPER_ADMIN · INSTALLER · DEVELOPER · DESIGNER · LIAISON · ADMIN_STAFF · SALES_STAFF

No transitions — a user's primary role is set directly. See [02 — Roles & Permissions](#).

ClientStatus

ACTIVE (default) · EXPIRED · SUSPENDED · CANCELLED

Free-form — any value can be set at any time. Not automatically driven by license expiry.

ClientType

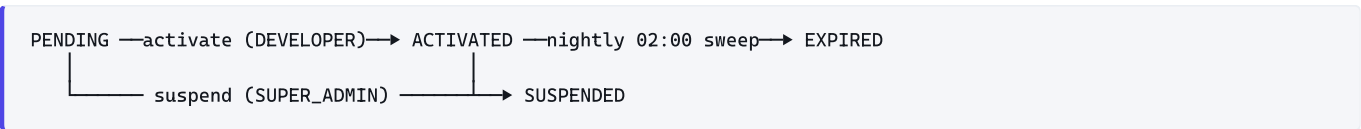
SOFTWARE (default) · ADVERTISING

LicenseType (product)

SUBSCRIPTION_MONTHLY · SUBSCRIPTION_ANNUAL · LIFETIME

LicenseStatus

PENDING (default) · ACTIVATED · EXPIRED · SUSPENDED

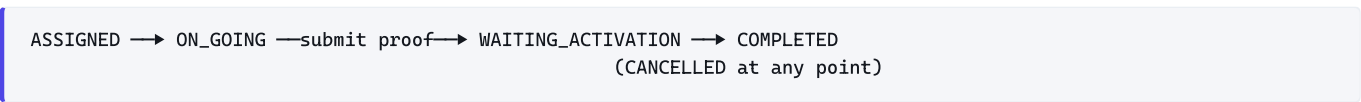


Enforced:

- Activating an already- ACTIVATED license → 409 License is already activated
- The expiry sweep only touches ACTIVATED licenses with a **past, non-null** expirationDate
- SUSPENDED and EXPIRED are not automatically reversible through the UI

JobStatus

ASSIGNED (default) · ON_GOING · WAITING_ACTIVATION · COMPLETED · CANCELLED



Enforced:

- Only INSTALLER may change status, and only on their own job

- Submitting proof **always** forces `WAITING_ACTIVATION`
- Assigning an installer **always** forces `ASSIGNED`
- `COMPLETED` requires the job's license — or **any** license of the job's client — to be `ACTIVATED`

Not enforced: the linear order itself. `ASSIGNED → COMPLETED` is accepted as long as the license condition holds.

JobOrderStatus

`DRAFT` (default) · `FINALIZED` · `ON_GOING` · `COMPLETED` · `CANCELLED`

No transition guard — any status can be set on save. Side effects:

Status	Labor earning (CCTV/Signage)	Inventory	Counted as revenue (Analytics)	In outstanding report
<code>DRAFT</code>	—	—	—	✓
<code>FINALIZED</code>	✓ created	—	✓	✓
<code>ON_GOING</code>	✓ ensured	—	—	✓
<code>COMPLETED</code>	✓ ensured	deducted	—	✓
<code>CANCELLED</code>	—	restored if it had been completed	—	—

JobOrderType

`SOFTWARE` (default) · `CCTV` · `SIGNAGE`

Determines the labor incentive formula and which extra fields appear.

DocType

`JOB_ORDER` (default) · `QUOTATION` · `INVOICE` · `RECEIPT`

Purely presentational + list-tab placement. Freely switchable via **Move to....** Converting a quotation to a job order forces `JOB_ORDER`.

DiscountType

`FIXED` (default) · `PERCENTAGE`

`PERCENTAGE` applies to the **subtotal** (sale price + materials).

PaymentMethod

`CASH` · `BANK_TRANSFER` · `GCASH` · `CHECK`

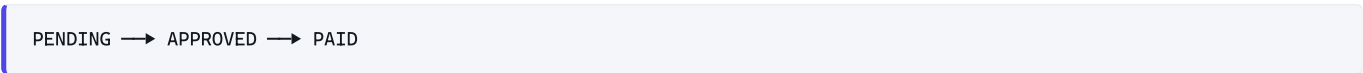
Payments have no status enum — a payment is either live or **voided** (`voidedAt` set). Voiding is one-way: a voided payment cannot be un-voided (`400 Payment is already voided`).

EarningType

`INSTALLATION` · `ACTIVATION` · `BONUS` · `COMMISSION`

EarningStatus

PENDING (default) · APPROVED · PAID



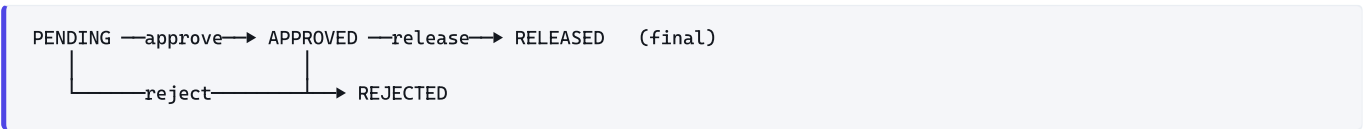
Strictly forward. Any other transition → 400 Cannot mark a {current} earning as {target}. Plus: 403 You cannot approve or pay out your own earning.

WithdrawalMethod

GCASH · MAYA · BANK_TRANSFER

WithdrawalStatus

PENDING (default) · APPROVED · REJECTED · RELEASED



Target	Allowed from
APPROVED	PENDING
REJECTED	PENDING , APPROVED
RELEASED	APPROVED

Violations → 400 Cannot mark a {current} withdrawal as {target}. Plus: 403 You cannot process your own withdrawal request.

Balance impact: PENDING , APPROVED and RELEASED all deduct from the available balance; REJECTED does not.

IncentiveStatus

PENDING (default) · APPROVED · PAID

Maps 1:1 onto the mirrored BONUS earning's status. No transition guard on the incentive itself — approve/pay set the value directly.

StockMovementReason

MANUAL_ADJUST · JOB_ORDER_DEDUCTION · JOB_ORDER_RESTORE

DevProjectStatus

NOT_STARTED (default) · IN_PROGRESS · PENDING · COMPLETED



`IN_PROGRESS` has two sub-states distinguished by `startedAt` : **running** (timestamp) vs **paused** (null).

Guards: see [12 — Dev Projects](#).

DevReportStatus

`PENDING` (default) · `REVIEWED`

Set to `REVIEWED` automatically when the tagged admin (or a Super Admin) posts feedback. There is no way back to `PENDING`.

Other value sets (not database enums)

Set	Values
Download lead platform	<code>ANDROID_APK</code> · <code>DESKTOP_PWA</code>
Finara lead status	<code>NEW</code> · <code>CONTACTED</code> · <code>CLOSED</code>
NENPOS client status	Free text, defaults to <code>ACTIVE</code>
Device token platform	<code>android</code> · <code>ios</code> · <code>web</code>
Mobile API env	<code>local</code> · <code>prod</code>

18 — API & Permission Matrix

Base URL: {host}/api

Notation: **SA** = SUPER_ADMIN , **AS** = ADMIN_STAFF , **SS** = SALES_STAFF , **LI** = LIAISON , **IN** = INSTALLER , **DV** = DEVELOPER , **DS** = DESIGNER , **any** = any authenticated user, **public** = no auth.

A request passes when **any** of the caller's roles (primary + additional) is in the allowed set.

Auth

Method	Path	Roles	Notes
POST	/auth/login	public	10 req/min per IP
POST	/auth/refresh	refresh token	Rotates the pair
POST	/auth/logout	any	Clears the stored refresh hash

Users

Method	Path	Roles
GET	/users/me	any
PATCH	/users/me	any
POST	/users	SA
GET	/users?role=	SA, AS
PATCH	/users/:id	SA
PATCH	/users/:id/activate	SA
PATCH	/users/:id/deactivate	SA
POST	/users/:id/roles	SA
DELETE	/users/:id/roles/:role	SA
PATCH	/users/:id/primary-role	SA

Clients

Method	Path	Roles
POST	/clients	SA, AS
GET	/clients?type=	SA, AS, LI, SS, DS
GET	/clients/:id	SA, AS, LI, SS, DS
PATCH	/clients/:id	SA, AS
DELETE	/clients/:id	SA, AS

Software Products

Method	Path	Roles
POST	/software-products	SA
GET	/software-products	any
GET	/software-products/:id	any
PATCH	/software-products/:id	SA
DELETE	/software-products/:id	SA

Licenses

Method	Path	Roles
POST	/licenses	SA
GET	/licenses	SA, DV
GET	/licenses/:id	SA, DV
PATCH	/licenses/:id/activate	DV only
PATCH	/licenses/:id/suspend	SA
PATCH	/licenses/:id	SA

NENPOS Clients

Method	Path	Roles
GET	/nenpos-clients	SA, AS
POST	/nenpos-clients	SA, AS
POST	/nenpos-clients/upload	SA, AS
PATCH	/nenpos-clients/:id	SA, AS
DELETE	/nenpos-clients/:id	SA, AS
DELETE	/nenpos-clients (all)	SA

Jobs

Method	Path	Roles	Notes
POST	/jobs	SA	
GET	/jobs	SA, IN, AS, LI, SS	Scoped to own jobs only when the primary role is IN; <code>?mine=true</code> is a no-op for other roles
GET	/jobs/calendar/month?month=&year=	SA, IN, AS, LI, SS	Not installer-scoped
GET	/jobs/calendar/day?date=	SA, IN, AS, LI, SS	Not installer-scoped
GET	/jobs/:id	SA, IN, AS, LI, SS	
PATCH	/jobs/:id/assign	SA, AS	Forces ASSIGNED , notifies

Method	Path	Roles	Notes
PATCH	/jobs/:id/status	IN only	Own job only
POST	/jobs/:id/proof	IN only	Own job only

Job Orders

Method	Path	Roles
POST	/job-orders (upsert)	SA, AS, LI, SS
GET	/job-orders	SA, AS, LI, SS
GET	/job-orders/by-job/:jobId	SA, AS, LI, SS, DS
GET	/job-orders/:id	SA, AS, LI, SS, DS
POST	/job-orders/:id/convert	SA, AS, LI, SS

Payments

Method	Path	Roles
POST	/job-orders/:id/payments	SA, AS, SS
GET	/job-orders/:id/payments	SA, AS, SS
POST	/payments/:id/void	SA, AS

Inventory

Method	Path	Roles
GET	/inventory	any
GET	/inventory/barcode/:code	any
GET	/inventory/:id/movements	any
POST	/inventory	SA, AS
PATCH	/inventory/:id	SA, AS
POST	/inventory/:id/adjust	SA, AS
DELETE	/inventory/:id	SA, AS

Earnings

Method	Path	Roles	Notes
POST	/earnings	SA	
GET	/earnings?mine=	any	All for SA/AS, own for everyone else; ?mine=true narrows SA/AS to their own
PATCH	/earnings/:id/approve	SA, AS	Not your own
PATCH	/earnings/:id/paid	SA, AS	Not your own

Withdrawals

Method	Path	Roles	Notes
GET	/withdrawals/balance	IN, DV, DS, LI, SS, AS	SA excluded
POST	/withdrawals	IN, DV, DS, LI, SS, AS	SA excluded
GET	/withdrawals?mine=	any	All for SA/AS, own for everyone else; ?mine=true narrows SA/AS to their own
PATCH	/withdrawals/:id/approve	SA, AS	Not your own
PATCH	/withdrawals/:id/reject	SA, AS	Not your own
PATCH	/withdrawals/:id/release	SA, AS	Not your own; optional proof URL

KPIs & Incentives

Method	Path	Roles
GET	/kpis/financial-summary	SA, AS
GET	/kpis/revenue-trend	SA, AS
GET	/kpis/team?month=&year=	SA, AS
POST	/kpis/manual	SA, AS
GET	/kpis/incentives?month=&year=	SA, AS
POST	/kpis/incentives/generate	SA, AS
PATCH	/kpis/incentives/:id/approve	SA, AS
PATCH	/kpis/incentives/:id/pay	SA, AS
GET	/kpis/designers/points?from=&to=	SA, AS
POST	/kpis/designers/sync	SA, AS
GET	/kpis/definitions/:role	SA, AS
POST	/kpis/definitions	SA, AS
PATCH	/kpis/definitions/:id	SA, AS
DELETE	/kpis/definitions/:id	SA, AS
GET	/kpis/dashboard	IN, DV, DS, SA, LI, SS, AS
GET	/kpis/incentives/mine	IN, DV, DS, LI, SS, AS (SA excluded)

Dev Projects

Method	Path	Roles
POST	/dev-projects	DV, SA
GET	/dev-projects	DV, SA, AS
GET	/dev-projects/developers	DV, SA, AS
GET	/dev-projects/reviewers	DV, SA, AS
GET	/dev-projects/active	DV, SA, AS
GET	/dev-projects/:id	DV, SA, AS

Method	Path	Roles
PATCH	/dev-projects/:id	SA only
POST	/dev-projects/:id/start	DV, SA
POST	/dev-projects/:id/pause	DV, SA
POST	/dev-projects/:id/resume	DV, SA
POST	/dev-projects/:id/stop	DV, SA
PATCH	/dev-projects/:id/progress	DV, SA
POST	/dev-projects/:id/reports	DV, SA
POST	/dev-projects/reports/:reportId/feedback	SA, AS

Beyond the role check, ownership is enforced: a developer may only touch their own projects.

Financial Reports

All under /reports/financial , guarded at the controller for **SA, AS, SS**.

Method	Path
GET	/reports/financial/collections?from=&to=
GET	/reports/financial/outstanding
GET	/reports/financial/client/:clientId
GET	/reports/financial/export

Notifications

Method	Path	Roles
GET	/notifications	any (own only)
GET	/notifications/unread-count	any
PATCH	/notifications/:id/read	any (own only)
PATCH	/notifications/read-all	any
POST	/notifications/device-token	any
DELETE	/notifications/device-token/:token	any

Events (SSE)

Method	Path	Roles
GET	/events	any

Uploads

Method	Path	Roles	Notes
POST	/uploads/images	any	Field <code>files</code> , ≤10 files, ≤10 MB each, images only
GET	/uploads/files/:filename	no guard	Serves the stored file

Company Profile

Method	Path	Roles
GET	/company-profile	any
PATCH	/company-profile	SA

Audit Logs

Method	Path	Roles
GET	/audit-logs	SA only

Backups / Restore / Reset

Entire controller is **SA only**.

Method	Path
GET	/backups
POST	/backups
POST	/backups/upload
POST	/backups/:filename/restore
GET	/backups/:filename/download
DELETE	/backups/:filename
GET	/backups/reset/modules
POST	/backups/reset/:moduleId

Download Leads

Method	Path	Roles	Notes
POST	/download-leads/send-code	public	5 req/min per IP
POST	/download-leads	public	10 req/min per IP
GET	/download-leads	SA, AS	
GET	/download-leads/finara	SA, AS	Live proxy

Standard error shapes

Status	When
400	Validation failure, illegal state transition, unknown property in the body, external-API failure
401	Missing/expired/invalid token, wrong credentials, wrong confirmation password, deactivated account
403	Role not permitted, acting on someone else's record, self-approval
404	Record not found
409	Uniqueness conflict (email, license key), or activating an already-activated license

Status	When
429	Rate limit exceeded
500	Unexpected failure (backup/restore SQL errors surface here with a reason)

Authorization test recipe

For each endpoint above:

1. **No token** → expect 401.
2. **Token of a role NOT in the allowed set** → expect 403.
3. **Token of each allowed role** → expect 2xx.
4. Where noted (jobs, dev projects, earnings, withdrawals, notifications) → test with a *different owner's* record and expect 403.

Expected: front-end route guards are convenience only. Every one of these checks must hold when called directly against the API.

19 — QA Test Checklists

A regression pass, module by module. Each item states the action and the **expected** result. Reference the chapter in brackets for the underlying rule.

Test accounts to prepare

#	Email	Primary role	Additional roles	Purpose
1	qa.super@...	SUPER_ADMIN	—	Full access, approvals
2	qa.admin@...	ADMIN_STAFF	—	Second approver, permission boundary
3	qa.sales@...	SALES_STAFF	—	Sales scope
4	qa.liaison@...	LIAISON	—	Liaison scope
5	qa.installer@...	INSTALLER	—	Field flow
6	qa.installer2@...	INSTALLER	—	Cross-ownership 403 tests
7	qa.dev@...	DEVELOPER	—	Activation + dev projects
8	qa.designer@...	DESIGNER	—	TMS KPI
9	qa.multi@...	SALES_STAFF	INSTALLER	Merged nav + merged permissions

Give account 1 a distinct base bonus (e.g. ₱20,000) so incentive maths is visibly per-user.

A — Authentication & session [03]

- ☐ A1 Valid login returns `accessToken`, `refreshToken` and a `user` with both `role` and `roles`
- ☐ A2 Wrong password, unknown email and deactivated account all return the **same** `401 Invalid credentials`
- ☐ A3 11 login attempts in 60 s → the 11th returns **429**
- ☐ A4 Access token expiry triggers a silent refresh; the user's action still completes
- ☐ A5 Reusing an already-rotated refresh token → `401 Access denied`
- ☐ A6 Logout, then refresh → `401 Access denied`
- ☐ A7 Deactivating a signed-in user breaks their **next API call**, not just their next login
- ☐ A8 Both successful and failed logins appear in the audit log; no plaintext password anywhere
- ☐ A9 Profile: new password without current password → 400; wrong current password → 401
- ☐ A10 Profile: changing email to an existing one → 409

B — Roles & navigation [02]

- ☐ B1 Each of the 9 accounts sees exactly the sidebar links in the navigation matrix
- ☐ B2 Account 9 sees the Sales nav plus an **Installer** section containing *My Jobs* only
- ☐ B3 Account 9 passes an `@Roles(INSTALLER)` endpoint (e.g. `PATCH /jobs/:id/status` on their own job)
- ☐ B4 Direct-URL navigation to a route the role lacks redirects instead of rendering
- ☐ B5 For every endpoint in [18], a disallowed role gets **403** and no token gets **401**
- ☐ B6 Setting a role as both primary and additional stores it once only

C — Clients [05]

- ☐ C1 Create a client; the code matches `CLT-[A-HJ-NP-Z2-9]{8}`
- ☐ C2 Duplicate client code is rejected
- ☐ C3 `SALES_STAFF` / `LIAISON` can list clients but get 403 on create/update/delete
- ☐ C4 `GET /clients/:id` includes `licenses[]` and `jobs[]`
- ☐ C5 Search matches code, business, owner, contact and email
- ☐ C6 Deleting a client with licenses/jobs — record the exact behaviour (see [20])

D — Products & licenses [06]

- ☐ D1 `ADMIN_STAFF` can view products but gets 403 on create/edit/delete
- ☐ D2 Non-trial license without a key → `400 License key is required for a non-trial license`
- ☐ D3 Duplicate license key → 409
- ☐ D4 Trial license: key matches `TRIAL-[A-HJ-NP-Z2-9]{4}-[A-HJ-NP-Z2-9]{4}`, `expirationDate` is null, `trialDays` defaults to 30
- ☐ D5 `trialDays` of 0 or 366 → 400
- ☐ D6 Only `DEVELOPER` can activate; `SUPER_ADMIN` → 403
- ☐ D7 Activating a 7-day trial sets expiry to activation + 7 days
- ☐ D8 Re-activating → `409 License is already activated`
- ☐ D9 Trial → full edit clears `trialDays` and `expirationDate`, keeps `ACTIVATED` and the token
- ☐ D10 Full → trial on an activated license → `400 An activated license cannot be changed back to a trial`
- ☐ D11 Back-date an activated license's expiry, run the 02:00 sweep → status becomes `EXPIRED`; `PENDING` / null-expiry licenses are untouched
- ☐ D12 NENPOS upload: template downloads, first sheet imports, blank Client Name → 400, blank Client ID → auto `NPC-` code

E — Installations [07]

- ☐ E1 Account 5 sees only their own jobs from `GET /jobs`
- ☐ E2 Account 6 changing account 5's job status → `403 You are not assigned to this job`
- ☐ E3 `SUPER_ADMIN` calling `PATCH /jobs/:id/status` → 403
- ☐ E4 Completing without any activated license → the documented 400 message
- ☐ E5 Activate **any** license for that client → completion now succeeds
- ☐ E6 Proof submission sets `WAITING_ACTIVATION`; re-submitting upserts one proof row and updates `capturedAt`
- ☐ E7 Reassigning an `ON_GOING` job resets it to `ASSIGNED` and notifies the new installer
- ☐ E8 Upload a PDF → 400; an 11 MB image → rejected; 11 images in one request → rejected
- ☐ E9 A job created without an installer shows **Unassigned**

F — Job orders & payments [08]

- ☐ F1 Step 3 (Payments) is disabled until the order is saved
- ☐ F2 Grand Total = `max(0, salePrice +  $\Sigma$ (qty×price) - discount)`; matches screen, API, print and payments panel
- ☐ F3 Percentage discount applies to the subtotal, not the sale price alone
- ☐ F4 A discount larger than the subtotal clamps the total to ₪0
- ☐ F5 Installer Labor is shown but excluded from the Grand Total
- ☐ F6 CCTV order finalized with no installer → `400 Assign an installer to the job before finalizing.` and **nothing is saved**
- ☐ F7 CCTV with 0 cameras / 0 rate → the cameras-and-rate 400

- ☐ F8 Signage with 0 labor % → the signage 400
- ☐ F9 Finalizing twice creates exactly **one** `INSTALLATION` earning
- ☐ F10 `SOFTWARE` type never creates a labor earning
- ☐ F11 Saving replaces line items wholesale; removed rows disappear
- ☐ F12 Convert a quotation → job created and linked, `docType` becomes `JOB_ORDER` ; converting again → `400 already linked`
- ☐ F13 Record a payment; balance drops. Void it; balance recovers and the row stays visible
- ☐ F14 Void with a 2-character reason → 400; voiding twice → `400 Payment is already voided`
- ☐ F15 `SALES_STAFF` can record a payment but has no Void button and gets 403 from the void API
- ☐ F16 **Overpayment** — record more than the balance and note the result (see [20])
- ☐ F17 Print each of the four doc types; letterhead, subtitle, filename prefix and signature label match [08]

G — Inventory [09]

- ☐ G1 Duplicate barcode → `400 That barcode is already used by another item`
- ☐ G2 Manual adjust below zero → `400 Stock cannot go below zero`
- ☐ G3 Complete a job order needing more stock than exists → succeeds, balance goes negative
- ☐ G4 Each change writes one ledger row with the correct running balance
- ☐ G5 Complete → un-complete nets to zero stock change (one `DEDUCTION` + one `RESTORE`)
- ☐ G6 Edit quantities on a completed order → only the difference moves
- ☐ G7 Barcode lookup ignores inactive items
- ☐ G8 Delete an item → job-order lines survive with a null inventory link

H — Earnings & withdrawals [10]

Run the worked example in chapter 10 end to end, then:

- ☐ H1 `PENDING` earnings do not raise the balance
- ☐ H2 Marking an earning `PAID` leaves the balance unchanged
- ☐ H3 `PENDING` → `PAID` on an earning → 400; `APPROVED` → `PENDING` → 400
- ☐ H4 Account 1 approving their own earning → 403; account 2 approving it → 200
- ☐ H5 Requesting more than the balance → `400 Insufficient balance. Available: Px, requested: Py.`
- ☐ H6 A `PENDING` withdrawal reserves its amount immediately
- ☐ H7 Rejecting a withdrawal returns the amount to the balance
- ☐ H8 `PENDING` → `RELEASED` (skipping approval) → 400
- ☐ H9 `RELEASED` → `REJECTED` → 400 (`RELEASED` is final)
- ☐ H10 Each status change produces a notification containing the peso amount
- ☐ H11 Release with a proof photo → the Proof column shows it; without → *No proof attached*
- ☐ H12 Non-admin users see only their own rows from the API
- ☐ H13 `SUPER_ADMIN` gets 403 on `GET /withdrawals/balance` and `POST /withdrawals`

I — KPI & incentives [11]

- ☐ I1 Complete 9 of 10 jobs for account 5 in one month → *Installation Completion Rate* = 90 %, score = 35
- ☐ I2 Actual above target never scores more than the full weight
- ☐ I3 Zero denominators produce 0, not `NaN`
- ☐ I4 Enter a manual value for an auto KPI → it overrides and is flagged manual
- ☐ I5 Total scores of 95.00 / 94.99 / 89.99 / 74.99 pay 100 / 75 / 50 / 0 %
- ☐ I6 Generate incentives twice for the same month → one incentive, one bonus earning

- ☐ I7 Approve an incentive → the mirrored earning becomes `APPROVED` and the balance rises
- ☐ I8 A user scoring below 75 → ~~₹~~0 incentive and **no** bonus earning
- ☐ I9 `SUPER_ADMIN` never appears in team KPI or incentive generation
- ☐ I10 Renaming a system-tracked KPI → `400 Cannot rename a system-tracked KPI`
- ☐ I11 Duplicate KPI name for a role → 409; a KPI for `SUPER_ADMIN` → 400
- ☐ I12 Designer sync with no token → the documented 400; with a bad token → the 401/non-JSON 400
- ☐ I13 Designer sync distributes points by weight; unmatched designers are written `applied: false`

J — Dev projects [12]

- ☐ J1 A developer sees and acts on only their own projects
- ☐ J2 `ADMIN_STAFF` can view all projects but gets 403 on start/stop/edit
- ☐ J3 Start project B while A runs → A auto-stops as `PENDING` with its time banked
- ☐ J4 Start → 60 s → pause → 60 s idle → resume → 60 s → stop = **2 minutes** total
- ☐ J5 Pause when not running → 403; resume when not paused → 403
- ☐ J6 Start a `COMPLETED` project → `403 This project is already completed`
- ☐ J7 Progress 100 % on a running project completes it and ends the run
- ☐ J8 Dropping a completed project below 100 % re-opens it as `PENDING`
- ☐ J9 Tag a non-admin as reviewer → 404
- ☐ J10 Tagged admin posts feedback → report `REVIEWED` + author notified; an untagged non-Super-Admin → 403
- ☐ J11 Assign a project to a non-`DEVELOPER` user → `404 Developer not found`

K — Financial reports [13]

- ☐ K1 Voided payments are excluded everywhere
- ☐ K2 The `to` filter includes the whole end day
- ☐ K3 The 6-month trend does not change when from/to narrows
- ☐ K4 `recentPayments` caps at 10
- ☐ K5 `CANCELLED` job orders never appear in outstanding; `DRAFT` ones do
- ☐ K6 A fully paid order drops off outstanding
- ☐ K7 CSV export row count matches the on-screen table
- ☐ K8 `LIAISON` / `INSTALLER` / `DEVELOPER` / `DESIGNER` get 403 on all four endpoints

L — Leads [14]

- ☐ L1 Second `send-code` inside 60 s → the cooldown 400
- ☐ L2 6th `send-code` inside a minute → 429
- ☐ L3 Code expires after 10 minutes
- ☐ L4 5 wrong attempts discard the code with *Too many attempts*
- ☐ L5 Resend unconfigured → `{ sent: false }`, lead still submittable as `emailVerified: false`
- ☐ L6 A used code cannot be reused
- ☐ L7 Emails are stored trimmed and lower-cased
- ☐ L8 Both public endpoints work with **no** Authorization header
- ☐ L9 Finara tab fetches only when opened; upstream failure shows a readable 400

M — Settings & administration [15]

- ☐ M1 Every `/settings` tab is 403/redirect for all non-Super-Admins
- ☐ M2 Password under 8 characters at user creation → 400
- ☐ M3 Duplicate email on create and on edit → 409
- ☐ M4 No users API response contains a password hash
- ☐ M5 Company profile change appears on the next print preview
- ☐ M6 Create a backup; it appears in the list with a non-zero size
- ☐ M7 Upload a `.txt` renamed to `.sql` ? → accepted by extension (document the result); a genuine `.txt` → 400 Only `.sql` backup files can be uploaded.
- ☐ M8 Upload with a path-traversal filename → the stored name is sanitised inside `backups/`
- ☐ M9 Restore with a wrong password → 401 Incorrect password
- ☐ M10 Module restore with no modules → 400; with an unknown id → 400; with modules absent from the dump → the documented 400
- ☐ M11 After a module restore, the scratch database `sdtmp_restore_*` is gone
- ☐ M12 Reset requires the password; a failed backup aborts the reset with nothing deleted
- ☐ M13 Reset never removes Users, Clients, Products, Company Profile, Inventory or KPI definitions
- ☐ M14 Reset `kpi` keeps KPI definitions and unlinks incentives from earnings
- ☐ M15 Audit log metadata masks every sensitive key

N — Mobile [16]

- ☐ N1 Tokens survive an app restart; a 401 refreshes silently
- ☐ N2 Admins see *Dashboard / Menu / Profile*; field staff see *Dashboard / My Jobs / Earnings / Profile*
- ☐ N3 An installer's job list contains only their own jobs
- ☐ N4 Photo + GPS proof submission lands on the web portal immediately
- ☐ N5 *Mark complete* is blocked with the same message as the web
- ☐ N6 The Server switcher persists across restarts and defaults to **prod**
- ☐ N7 A non-admin deep-linking to `/admin/clients` is redirected to the tabs
- ☐ N8 Withdrawal requests obey the same balance rules

O — Cross-cutting

- ☐ O1 Every mutation appears in the audit log with the right action name and actor
- ☐ O2 SSE: a change made in one browser updates another browser's dashboard and sidebar dot without a reload
- ☐ O3 Posting an unknown property to any endpoint → 400 (`forbidNonWhitelisted`)
- ☐ O4 301 requests in 60 s from one IP → 429
- ☐ O5 Sidebar collapse state survives a reload; the mobile drawer closes on navigation
- ☐ O6 Light and dark themes are both legible on every page, and the theme persists
- ☐ O7 Peso amounts render with the ₱ symbol and thousands separators throughout
- ☐ O8 Every list has a distinct empty state, no-match state and error state (see each chapter for the exact strings)

Smoke test (15 minutes)

The shortest path that exercises the whole chain:

1. Sign in as Super Admin → create a client
2. Create a software product → generate a **trial** license for that client

3. Create an installation job for the client, assign account 5
4. Sign in as account 7 (developer) → activate the license with a fingerprint
5. Sign in as account 5 (mobile or web) → start the job, submit proof → **WAITING_ACTIVATION**
6. Mark complete → succeeds because the license is activated
7. Super Admin → create a **CCTV** job order on that job with 4 cameras @ ₱500, two line items, 10 % discount → finalize → verify Grand Total and that a ₱2,000 **INSTALLATION** earning appeared for account 5
8. Record a partial payment → check the balance on the order and in Financial Reports
9. Approve account 5's earning (as Super Admin) → account 5 requests a withdrawal → approve → release with proof
10. Generate incentives for the current month → check the KPI scorecard and the mirrored bonus earning
11. Settings → create a backup → confirm it lists and downloads

20 — Known Gaps & Troubleshooting

Read this **before filing a bug**. Everything below is present in the shipped code and was found while writing this manual. Each entry states what the code actually does, so QA can decide whether it is a defect or intended behaviour and file it with evidence rather than re-discovering it.

Permission mismatches

Front-end route guards are wider than the matching API guard, so the page loads and then fails with 403.

G1 — `/audit-logs` admits `ADMIN_STAFF`, the API does not

- Admin-web route allows `SUPER_ADMIN` and `ADMIN_STAFF`, and the `ADMIN_STAFF` sidebar has an **Audit Logs** link.
- `AuditLogsController` is decorated `@Roles(SUPER_ADMIN)` at class level.
- **Result:** an `ADMIN_STAFF` user clicks *Audit Logs*, the page renders, and `GET /api/audit-logs` returns 403 → **"Failed to load audit logs."**
- Fix is one line either way: widen the controller, or remove the role from the route and the nav.

G2 — `/licenses` admits `ADMIN_STAFF`, the API does not

- Admin-web route allows `SUPER_ADMIN`, `DEVELOPER` and `ADMIN_STAFF`.
- `GET /api/licenses` allows `SUPER_ADMIN` and `DEVELOPER` only.
- **Result:** an `ADMIN_STAFF` who navigates directly to `/licenses` sees **"Failed to load licenses."** on the Licenses tab, while the **NENPOS Clients** tab on the same page works (that API *does* allow `ADMIN_STAFF`).
- Note there is no `ADMIN_STAFF` sidebar link to `/licenses`, so this is only reachable by direct URL.

G3 — Mobile: `/admin/*` admits `SALES_STAFF`, the Menu tab does not

- `mobile/app/admin/_layout.tsx` allows `SUPER_ADMIN`, `ADMIN_STAFF` and `SALES_STAFF`.
 - The **Menu** tab only renders for `SUPER_ADMIN` and `ADMIN_STAFF`.
 - **Result:** a sales user has no UI path into the admin screens even though the routes would accept them (deep links work).
-

Scoping gaps

G4 — Job calendar endpoints are not installer-scoped

`GET /api/jobs/calendar/month` and `/calendar/day` return **all** jobs for the period regardless of who asks, while `GET /api/jobs` scopes installers to their own jobs. An installer's dashboard calendar therefore shows other installers' appointments.

G5 — `GET /api/jobs?mine=true` is a no-op for non-installers

The controller sets the user id when the caller **has** the `INSTALLER` role or passes `mine=true`, but `JobsService.findAll` only applies the filter when the caller's **primary** role is `INSTALLER`. Any other role passing `?mine=true` still receives the full list. (`/earnings?mine=true` and `/withdrawals?mine=true` work correctly — they filter on the user id directly.)

Business-rule questions to confirm with the product owner

These are not obviously bugs, but they will surprise users.

G6 — Revenue counts only `FINALIZED` job orders

`getFinancialSummary` and `getRevenueTrend` filter on `status = FINALIZED`. An order that has moved on to `ON_GOING` or `COMPLETED` is **excluded** from Total Revenue, Monthly Growth and the 6-month trend on the Dashboard and Analytics pages. Revenue therefore *drops* as work progresses.

They also sum `salePrice` only — materials and discounts are ignored, so the Analytics revenue does not match the invoice grand totals used by Financial Reports.

G7 — Overpayment is not blocked

`PaymentsService.recordPayment` performs no check that `amount ≤ balance`. Recording ₱10,000 against a ₱5,000 order produces a – ₱5,000 balance, which then flows into the outstanding report (as a negative, though rows with `balance <= 0.005` are filtered out, so it simply disappears from that list).

G8 — KPI weights are not validated to 100

Nothing checks that a role's KPI weights sum to 100. The KPI Settings screen shows a **Total weight** readout but does not enforce it. A role whose weights sum to 120 can reach a total score above 100 and will always land in the top incentive tier.

G9 — Job order status transitions are unguarded

Any `JobOrderStatus` can be set on any save — `COMPLETED → DRAFT` is accepted. The side effects (inventory, labor earning) are reconciled correctly, but there is no workflow enforcement.

G10 — Client status is decorative

`ClientStatus` is never driven by license expiry and never blocks anything. A `CANCELLED` client can still receive new jobs, job orders and licenses.

G11 — `IncentiveStatus` has no transition guard

Unlike earnings and withdrawals, an incentive can be moved directly to `PAID` without being approved. The mirrored earning is set to match, bypassing the earning's own forward-only guard.

Incomplete features

G12 — MFA is scaffolded but not implemented

`users.mfa_enabled` and `users.mfa_secret` exist, the Users screen shows an MFA column, and `mfaSecret` is on the audit-log scrub list — but `AuthService.login` contains **no OTP step**. Enabling MFA on a user changes nothing about how they sign in.

G13 — Client deletion has no dependency check

`ClientsService.remove` calls `prisma.client.delete` directly. A client with licenses, jobs or job orders will fail on a foreign-key constraint and surface as a raw 500 rather than a friendly *"This client still has ..."* message.

Environment / infrastructure notes

E1 — Lead OTP codes live in process memory

Verification codes are held in a `Map` inside `DownloadLeadsService`. A restart or redeploy invalidates every pending code, and codes are not shared between instances if the API is ever scaled horizontally. Visitors simply request a new code. **Not a defect.**

E2 — Backups fall back to a JavaScript dumper

If no `mysqldump` binary is found (`ENOENT`), the service logs *"mysqldump binary not found (...); using the built-in JS dumper"* and produces the dump in pure JavaScript. Both paths write `DROP TABLE IF EXISTS` , so either dump restores cleanly. Set `MYSQLDUMP_PATH` to force a specific binary.

E3 — Finara production has returned 404 for the leads export

The Finara tab surfaces this as `Finara API returned HTTP 404.` — an upstream deployment issue with `/api/leads/export` , not a METRIQA defect.

E4 — Migrations run at boot and never crash the app

`main.ts` calls `npm run prisma migrate deploy` before bootstrapping. A failure is logged (`prisma migrate deploy failed: ...`) and the app boots anyway, so a schema-drifted database shows up as 500s on affected endpoints rather than a crash-loop.

E5 — Development ports differ from production

`npm run start:dev` forces `PORT=3002` ; production defaults to `3000` . The mobile app's local default points at `:3001` . Confirm the port before reporting "cannot reach the API".

Error message catalogue

Use this to map a symptom to its source.

Authentication & authorization

Message	Status	Cause
Invalid credentials	401	Unknown email, wrong password, or deactivated account
Access denied	401	Refresh token missing, rotated or cleared by logout
Account is inactive or no longer exists	401	User deactivated or deleted mid-session
Incorrect password	401	Wrong confirmation password on restore/reset
Current password is incorrect	401	Profile password change
Current password is required to set a new one	400	Profile password change
Forbidden resource	403	<code>RolesGuard</code> rejected the role
ThrottlerException: Too Many Requests	429	Rate limit

Ownership & self-action

Message	Status
You are not assigned to this job	403
You cannot approve or pay out your own earning.	403
You cannot process your own withdrawal request.	403
You do not have access to this development project	403
You were not tagged on this report	403
Not your notification	403

State transitions

Message	Status
Cannot mark a {x} earning as {y}.	400
Cannot mark a {x} withdrawal as {y}.	400
Cannot complete this job: the license has not been activated yet. ...	400
License is already activated	409
An activated license cannot be changed back to a trial	400
This order is already linked to an installation job.	400
Payment is already voided	400
This project is already being worked on	403
This project is already completed	403
This project is not currently running	403
This project is not currently paused	403

Validation & conflicts

Message	Status
A user with this email already exists / Email already in use by another account	409
A license with this key already exists	409
License key is required for a non-trial license	400
KPI "{name}" already exists for {ROLE}	409
Cannot rename a system-tracked KPI	400
Role {X} does not have KPI tracking	400
That barcode is already used by another item	400
Stock cannot go below zero	400
Client Name is required.	400
Insufficient balance. Available: ₧x, requested: ₧y.	400
Assign an installer to the job before finalizing.	400
Enter the number of cameras and rate per camera before finalizing.	400
Signage labor is zero – check the total price and labor % before finalizing.	400
Only image files (JPEG, PNG, GIF, WEBP) are allowed	400
Only .sql backup files can be uploaded.	400
No file uploaded.	400

Backups, restore, reset

Message	Status
Backup failed: {reason}	500
Auto-backup failed – reset aborted for safety. {reason}	400
Restore failed: {reason}	500

Message	Status
Select at least one module to restore.	400
Unknown module: {id}	400
None of the selected modules had tables in this backup.	400
Backup not found	404

External integrations

Message	Status
TMS_KPI_API_TOKEN is not configured on the server.	400
Could not reach the TMS KPI API.	400
TMS KPI API rejected the token (401 Unauthorized).	400
TMS KPI API returned HTTP {status}.	400
TMS KPI API did not return JSON – the token may be invalid or lack the kpi:read ability.	400
FINARA_API_KEY is not configured on the server.	400
Could not reach the Finara API.	400
Finara API rejected the key (401 Unauthorized).	400
Finara API returned HTTP {status}.	400
Finara API did not return JSON – the API key may be invalid.	400

Lead verification

Message	Status
Please wait a minute before requesting another code.	400
Verification code expired – request a new one.	400
Too many attempts – request a new code.	400
Incorrect verification code.	400

Common UI error strings and what they mean

Every one of these means "the underlying query returned an error" — check the network tab for the real status.

String	Page
Failed to load clients.	Clients
Failed to load products.	Products
Failed to load licenses. / Failed to load records.	Licenses / NENPOS tab
Failed to load your jobs. / Failed to load installations.	Installations
Failed to load job orders.	Job Orders
Could not load job order. / Parent job not found.	Job Order editor
Failed to load earnings.	Earnings
Failed to load withdrawal requests.	Withdrawals

String	Page
Failed to load inventory. / Failed to load history.	Inventory
Failed to load development projects. / Failed to create project. / Failed to post report.	Dev Projects
Failed to load KPI definitions. / Failed to save.	KPI Settings
Failed to load leads.	Download Leads
Failed to load audit logs.	Audit Logs
Failed to load company profile. / Failed to load backups. / Failed to load modules.	Settings
Failed to load the team.	Users
Could not update the client. Try again.	Clients
Could not create the client. Try again.	Dashboard quick action
Could not convert this order. Try again.	Job Order editor
Failed. Try again.	Installations